



M256 Unit 3
UNDERGRADUATE COMPUTING

Software development with Java



Creating a conceptual
model

Unit
3

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)870 333 4340, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; email ouwenq@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007. Second edition 2008.

Copyright © 2007, 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 5476 6

2.1

CONTENTS

1	Introduction	5
1.1	Conceptualising the system domain as classes	5
1.2	The aims of this unit	5
1.3	Studying this unit	6
2	Developing the conceptual model	7
2.1	Main activities in developing a conceptual model	7
2.2	The role of the conceptual model	8
2.3	The conceptual model in the context of the wider system	8
3	Classes	10
3.1	Terminology and notation	10
3.2	Identifying classes and attributes	12
3.3	Identifying classes and attributes for the Hospital System	19
3.4	Generalisation relationships	23
3.5	Abstract classes	25
3.6	Class or attribute?	28
4	Associations	30
4.1	What is an association?	30
4.2	Identifying associations	31
4.3	Multiplicity	34
5	Modelling events	41
5.1	Criteria for modelling events	41
5.2	Summary of criteria	45
6	Invariants	47
6.1	What is an invariant?	47
6.2	Remaining invariants for the Hospital System	51
6.3	Use of the term 'constraint'	56
7	Derived attributes and associations	57
7.1	Using invariants to derive attributes	57
7.2	Using invariants to derive associations	59
8	Conceptual model for the Hospital System	64
9	Summary	68
	Glossary	69
	References	71
	Index	72

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In *Unit 1* you were given an overview of the software development process and the object-oriented approach we will be using in M256. You learnt that software development is made up of phases, which are often organised into cycles. The first of these phases is requirements specification, which was the subject of *Unit 2*. In this unit you will explore how to analyse the requirements document (the output from the requirements specification phase) to arrive at an understanding of how the proposed software system will be structured.

1.1 Conceptualising the system domain as classes

The first step in structuring the system is to model the system domain in terms of a collection of classes and the relationships between them. These classes are *conceptual* classes – not the software classes that will eventually form part of a working system. They are modelling elements, corresponding to real-world entities mentioned in the requirements document. The relationships between them represent connections that are important in the system domain. The classes and relationships should all be ones that the client who has commissioned the software can easily relate to. You will see that, although building this initial model takes expertise, the result ought to be meaningful to someone who is non-technical but who knows about the system domain. So, for instance, if the requirement was for a system to administer a chain of shops, we might have classes such as *Shop*, *Employee*, *Manager* and so on, and relationships such as ‘works at’, and ‘manages’. This picture of the system domain as a collection of related conceptual classes will be referred to as the **conceptual model**.

Recall that the system domain is the real-world context specific to the software system being developed.

Once you have drawn up the conceptual model, you will use it as the basis for a series of models of the structure of the system, which will lead eventually to a specification of the software that is to be implemented.

1.2 The aims of this unit

Starting with the requirements document for the Hospital System developed in *Unit 2*, the main aim of this unit is to construct a conceptual model for that system. In drawing up this model, the unit:

- ▶ sets out some of the key activities and concepts that are involved in developing a conceptual model (Section 2);
- ▶ introduces techniques for analysing the requirements document to identify conceptual classes and their attributes (Section 3);
- ▶ outlines the different types of relationship that can exist between classes, and how to identify them (Sections 3 and 4);
- ▶ describes how the model can include other kinds of information which is discovered from analysis, such as events and constraints (Sections 5 and 6);
- ▶ shows you how to identify information that can be derived from other information in the model (Section 7);

- ▶ in the process of doing all the above, develops a conceptual model for the Hospital System (Section 8).

UML-like class and object diagrams will be used to help you develop and think about the conceptual model. These diagrams can be used to describe the *software* classes and objects that will eventually be implemented in code, but they can equally well be used for *conceptual* models that deal with entities from the system domain. The UML is flexible enough for both purposes.

SAQ 1

Section 4 of *Unit 1* listed a number of activities involved in an object-oriented approach to software development. The first two of these were requirements specification and developing a conceptual model. What were the others?

ANSWER.....

- ▶ Developing dynamic models
 - ▶ Developing a user interface
 - ▶ Detailed design and implementation
 - ▶ Testing
 - ▶ Maintenance
-

1.3 Studying this unit

This unit contains a number of SAQs and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on. This unit contains no practical exercises requiring the use of a computer. Some exercises will require you to produce diagrams. You could use a drawing package for this, or simply hand-draw the diagrams.

Unit 3 is a little longer than average and may take you slightly more time to study than the *Study Calendar* indicates. However, *Unit 4* will be correspondingly shorter. We expect you to spend about half your time on Sections 3 and 4 of this unit, which cover the basic ideas involved in building a conceptual model. Sections 5, 6 and 7 are shorter, but cover concepts that you might find more challenging. The final section simply brings together the work you have done earlier in the unit.

In studying this unit, you will need to refer to the *Handbook* and we suggest you keep it by your side as you work.

2

Developing the conceptual model

In this section you will learn about:

- ▶ the activities involved in developing a conceptual model;
- ▶ the role and purpose of the conceptual model;
- ▶ the position of the conceptual model in the wider context of the system that is being developed.

2.1

Main activities in developing a conceptual model

The process of constructing a conceptual model starts with an analysis of the requirements document, to identify those entities which are of key significance to the system, excluding any that are irrelevant or peripheral to its core functionality. In the case of the Hospital System, for example, key entities will include things such as patients and wards.

One category of entity that will be excluded at this stage is that which deals with details of input and output – the way in which information will be provided by and to human users of the systems. In *Unit 1* you saw that it is standard practice to separate the development of the user interface from that of the core system, and keep the two as independent of one another as possible. This is because it is quite likely that the user interface will need to be changed after the software has been delivered, for example to accommodate additional types of user or to take advantage of improved technology. This will be much easier to do if the user interface can be altered without requiring changes to the core system. Conversely, changes to the implementation of the core system should be possible while still keeping the user interface the same.

Having identified entities that are of key significance to the system, we need to identify their properties and the various relationships between them. For example, patients are likely to have properties such as their name and age, and to be related to wards by virtue of the fact that each patient is on a particular ward.

It is also necessary to identify **constraints on the system domain** (i.e. limitations on the entities, their properties and relationships) and decide what conditions such constraints will impose on the model.

The completed conceptual model will consist of a diagram representing the system domain, together with text describing important features of the model.

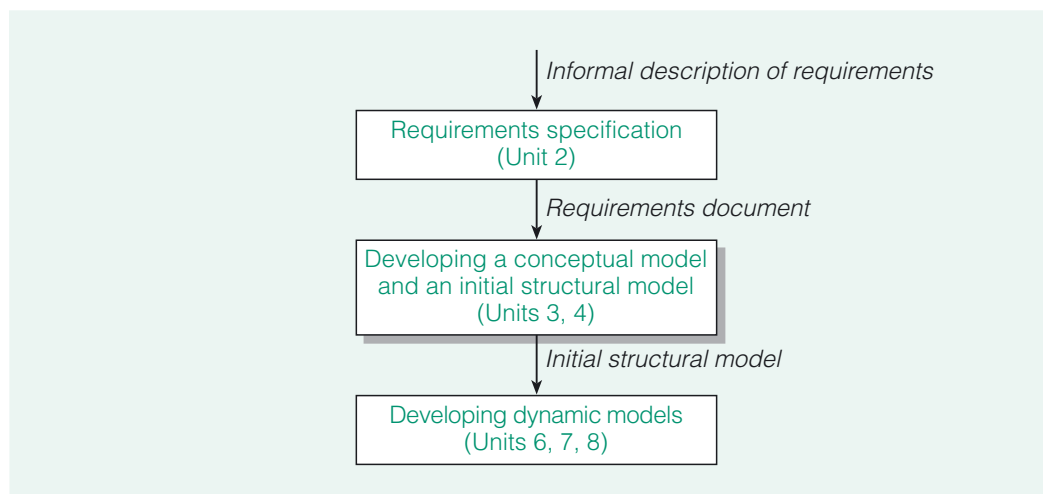
In summary, the process which takes you from the requirements document to the conceptual model involves the following activities.

- 1 Identification of those real-world entities described in the requirements document which are concerned with the core functionality of the system, leaving out considerations of the user interface. These will become the conceptual classes in the model.
- 2 Identification of the properties of the entities and the relationships between the entities.
- 3 Identification of the conditions that will be imposed on the model.
- 4 Representation of the model using diagrams and associated text.

2.2 The role of the conceptual model

At the very heart of object-oriented programming is the philosophy that software objects can usefully correspond to real-world entities. This is why we create a conceptual model that ‘looks’ object-oriented: it includes *modelling elements* referred to as ‘classes’, which have instances referred to as ‘objects’ and so on. Having a conceptual class such as *Shop*, for example, makes it sensible to consider having a *Shop* class as part of the software. Thus the conceptual model will provide an initial basis for the structure, or architecture, of the software – the classes and objects that make up the system and the way in which they fit together. That is, the conceptual model gives rise to an **initial structural model** for the system under development. It is important to recognise that whilst on paper the conceptual model and the initial structural model may ‘look’ the same, they are not. They model different things, from different perspectives. A conceptual model is a representation of the system domain, whilst an initial structural model is a plan – an early model – of the software itself.

The initial structural model does not deal with the behaviour of the system – the way in which objects interact. Behaviour is addressed by designing *dynamic* models, which you will study in later units. The way in which the conceptual and initial structural models fit into the software development process is illustrated in Figure 1.



The full diagram, of which Figure 1 is a part, is given in Section 1 of the *Handbook*.

Figure 1 The place of the conceptual and initial structural models in the software development process

The initial structural model is what is taken forward to the next stage of development. It will become the basis for designing the software system, and as you go along many of the conceptual classes you identify initially will crystallise into corresponding software classes (written in Java in this course). But it is important to recognise that not all the conceptual classes will necessarily have software counterparts, and also that the software is likely to include classes which had no conceptual equivalent but which are needed in order to achieve a good software design.

2.3 The conceptual model in the context of the wider system

To place the conceptual model in context, it will help to review briefly some particularly relevant terms that were introduced in *Units 1* and *2*. The broad real-world context in which the software under development is to operate is known as the *business area*. This encompasses all the entities and processes, systems and subsystems that contribute to the activities of a business or other organisation, as well as the communication

between them. For example, the Hospital System could be viewed as one subsystem in the general business area which covers everything that goes on in hospitals: the dispensing of medicines, the training of nurses and doctors, and so on. Each of these activities may constitute a subsystem, which may or may not be computerised, but which will need to communicate with other subsystems in the broader area.

The *system domain* is the particular sub-part of a business area for which a software system is being developed. The entities in the system domain are those of potential significance for that system. For example, you know from the requirements document that the entities which form part of the system domain for the Hospital System include doctors, patients and wards, but not nurses and medicines.

Finally, that part of the software that is separate from the user interface and that contains objects corresponding to those in the system domain is referred to as the *core system*.

In the real world, a system which dealt with the admission and treatment of patients might need to communicate with systems such as those controlling nursing shifts and the stock control of medicines. M256 will not be concerned with communication between separate systems; it will consider only the communication between the system and its human users – the staff who will be interacting with the system.

SAQ 2

A computer system is required for booking badminton and squash courts in a sports centre which provides a number of other facilities for its members.

- (a) Of what general business area might this system domain be a part?
- (b) Suggest another system domain from the same business area for which a separate computer system might be needed.

ANSWER.....

There is no single correct answer to these questions, and your solutions may be different from ours but equally valid.

- (a) The system domain could be seen as belonging to the general business area of sports-centre management or administration; alternatively you could have seen it as belonging to a broader business area such as administration of leisure facilities.
- (b) Taking the business area of sports-centre administration, other required systems might include things such as a membership database, a staff records system or a facilities maintenance system.

In this section you have learnt about what is involved in the process of creating a conceptual model and how this model forms the initial structural model for the eventual software system. You also learnt about the conceptual model in the context of the wider system.

3

Classes

In this section, you will learn ways of using a requirements document as the basis for identifying the conceptual classes needed to represent entities from the system domain.

First, though, we will present some terms and notation associated with the process of analysing the requirements document. Several of these will be familiar to you from your knowledge of software implementation, although here they will be used in a slightly different context, to describe conceptual entities in the system domain.

3.1 Terminology and notation

A **class** models a category of real-world entities from the system domain. Although these may be tangible real-world 'things', they may also be intangible things such as events, roles and organisational units. Whether the entities are tangible or not, the modelling elements that represent them are referred to as **objects**.

Objects of a particular class may have properties called **attributes**, corresponding to properties of their real-world equivalents. For example, an object representing a car entity may have an attribute `colour` that corresponds to the colour property of a car. (Note that, since objects of a particular class all have the same attributes, we also say that the class has those attributes.)

At any given time an attribute of an object has a **value**; that is, specific information that the object holds in respect of that attribute. For example, the `colour` attribute of an object representing a particular car may have the value `red`.

Objects may also have **links** to one or more other objects, representing connections between real-world entities. If objects of one class can be linked to objects of another class, there is an **association** between the classes.

Here is an example. Imagine a very simple system domain, where all we want to do is record information about a number of vehicles and their drivers, as follows:

- ▶ the number of wheels each vehicle has, and its colour;
- ▶ the name of each driver;
- ▶ the driver who drives a particular vehicle or vehicles.

In modelling this system domain there are two classes, `Vehicle` and `Driver`. `Vehicle` has two attributes, `numberOfWheels` and `colour`. `Driver` has one attribute, `name`. If a particular driver drives a particular vehicle, the corresponding objects are linked. The classes `Driver` and `Vehicle` are associated, because objects of the two classes may be linked.

The UML provides notation for representing classes and associations using **class diagrams**, and objects and links using object diagrams. You saw some examples of object diagrams in *Unit 1*. There they were being used to show software objects – in other words instances of software classes – but in this unit we will use them to represent entities in the system domain.

The class diagram in Figure 2 shows the two classes `Vehicle` and `Driver`, their respective attributes, and the association between them, which we have called `drives`.

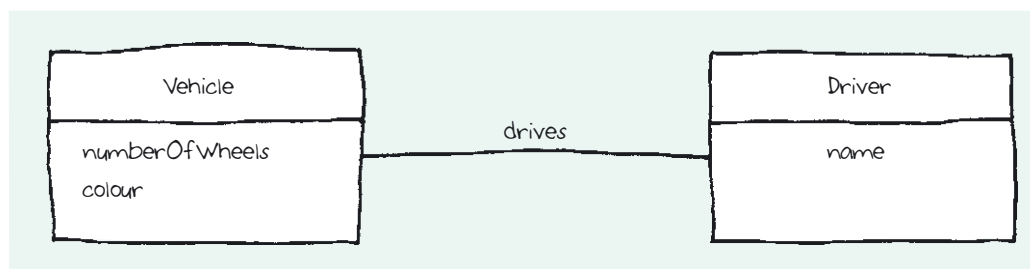


Figure 2 A class diagram

You might think that the name *drives* in Figure 2 implies that the association has a direction (i.e. a driver drives a vehicle), but it does not. We could equally well have called the association *isDrivenBy* instead (i.e. a vehicle is driven by a driver).

As you learnt in *Unit 1*, Figure 3 is an object diagram. It shows particular objects of the two classes shown in Figure 2 and the links between them that are instances of the association *drives*.

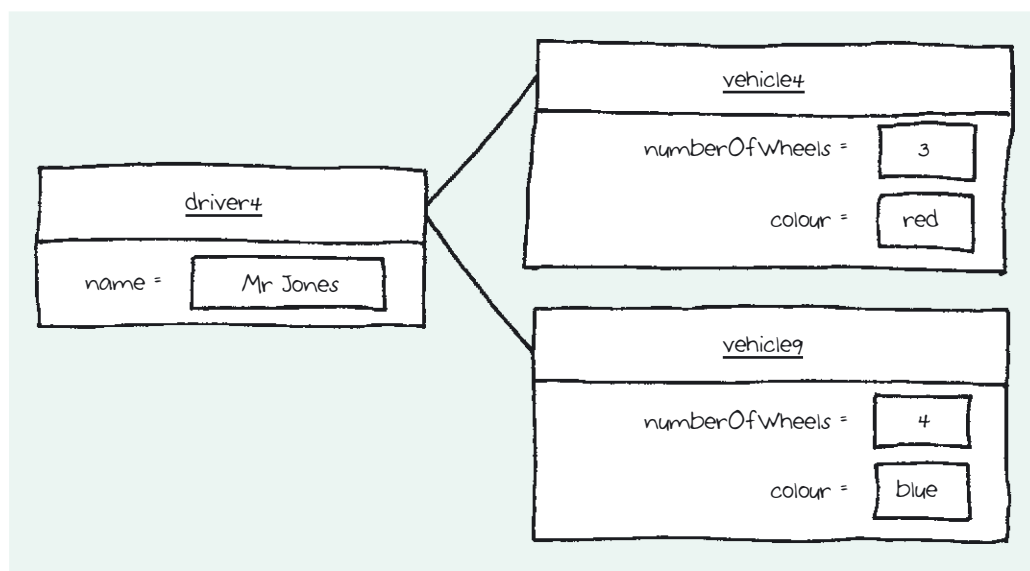


Figure 3 An object diagram

In this example an imaginary driver represented by an object identified as *driver4* drives vehicles represented by objects identified as *vehicle4* and *vehicle9*. We have also invented some values for the attributes.

It is often, but not always, the case that the attributes identified at this stage are later implemented as instance variables, but note that, at the conceptual modelling stage, no assumption is made as to whether attribute values will eventually be implemented in the software by strings, integers or any other type. Consequently, the value of the name attribute for the object *driver4* is not enclosed in quote marks as in "Mr Jones", since this would imply that it was to be implemented by a string.

When you met object diagrams in *Unit 1*, they were not conceptual. They represented actual instances of Java classes (hence the values for name attributes, like "Rosie Webster" were enclosed in quote marks as they were represented by strings in the code). You also saw that links were implemented by instance variables, which held references to other objects. Informally you can think of this as one object 'knowing' about another.

However, the links in Figure 3 are *conceptual* links, and at this stage no conclusions can be drawn about whether *Driver* objects 'know' about *Vehicle* objects, *Vehicle* objects 'know' about *Driver* objects, or both these things. In fact, for all that is known,

The inclusion of attribute names in the classes which make up a class diagram is optional. In this course we will henceforth leave them out and instead, as you will see, include them in textual class descriptions.

Dynamic modelling will be dealt with in *Unit 6*.

neither might apply! The conceptual links just mean that the real-world entities represented by the objects are related in some way. Only when deciding how the system will perform its tasks (i.e. during dynamic modelling), will it be possible to consider how the *software* objects may need to interact,

There is an important relationship between the kinds of diagram in Figures 2 and 3. The first – the class diagram – is *general*, and the second – the object diagram – is *specific*. A class diagram, as depicted in Figure 2, represents *all* the objects of the classes shown, and the association represents *all* the links between objects of the two classes. The object diagram in Figure 3 depicts *particular* objects and the *particular* links between them.

When you go from the general to the specific like this, you refer to the specific cases as **instances** of the general category. Thus an object is an instance of a class, and in exactly the same way a link is an instance of an association.

This will sound very familiar from programming – after all, you already know that classes have instances – but remember that we are not talking about software yet. At the moment we are still discussing categories and entities from the system domain.

The next subsection takes forward the idea of a class diagram. We begin by examining in more detail how to identify classes, attributes and associations from a requirements document.

3.2 Identifying classes and attributes

The analysis process starts with the identification of a set of conceptual classes – the categories of things which are of significance in the system domain.

There is not usually a single correct choice of classes to model a given system domain. Different people or different software development teams starting with the same requirements will not necessarily come up with the same set of conceptual classes. But good choices – those that accurately model the system domain – are crucial to the software development process. Inappropriately chosen classes may add complexity to later phases of the development cycle, and could make the software difficult to maintain and/or extend.

When identifying appropriate classes, a good understanding of the system domain is important. The software developer often gains this through discussion with the client, assumed to be an expert in the area. Possible classes for inclusion may thus emerge during the course of the requirements specification process. You are in the rather artificial situation of having no client with whom to discuss the requirements. However, there are a number of other techniques that you can use to help identify appropriate classes from a requirements document.

Textual analysis

Because a class is conceptually a set of objects of the same kind, and objects represent 'things' in the system domain, nouns and noun phrases in the requirements document can be used as an aid to identifying possible classes.

Below is an extract from the requirements document for a DVD Library System with the nouns and noun phrases underlined. We have not included the acceptance tests, as these are derived from the use cases, so do not add anything to the description of the system.

A **noun phrase** is a phrase (a sequence of words like 'number of copies') that functions as a noun – and which can be used anywhere that a noun could be used.

System domain

The DVD Library holds DVDs of a number of films. It may have any number of copies of each film. The system should, for each film, hold the film's title; no two films have the same title.

A library member can borrow any DVD of a film that is not currently on loan. However, no member can have more than six DVDs on loan at once. A borrowed DVD must be returned within three days. When a library member returns a DVD it immediately becomes available for loan to other members.

Each DVD has a unique identifying number.

Each member of the library has a unique membership number; the system also holds each member's name.

Use cases

The DVD Library System will provide support to the library by enabling the library staff to do the following:

- A Get Number Available.** The staff member identifies the film. The system displays the number of copies available.
- B List Films.** The staff member identifies the member. The system displays a list of the film titles and return dates for the DVDs that the member has on loan.
- C Borrow DVD.** The staff member identifies the DVD and the member who is borrowing it. The system records the fact that this copy of the film is no longer available for loan, the fact that it is on loan to the member identified, and the date on which it was borrowed.
- D Return DVD.** The staff member identifies the DVD. The system records the fact that this copy of the film is now available for loan and that it is no longer on loan to the member who returned it.

Choosing the nouns and noun phrases is not an exact science. For example, in the second sentence of the above description, the object of the verb 'have' is the noun phrase 'any number of copies of each film'. Instead of underlining this phrase as a whole we have chosen to identify 'number of copies' and 'film' separately within the phrase. In doing this we are intuitively deciding that both copies and films are likely to be of importance. Another example is the noun 'loan', which could be treated as part of the phrase 'is on loan to the member' (describing the status of a DVD) rather than as a noun in its own right. This is only the first stage of identifying classes. We will shortly analyse the **candidate classes** in more detail.

Some of the nouns and noun phrases underlined are duplicates, and some simply provide different ways of referring to the same thing. For example, 'member' and 'library member' clearly refer to the same kind of entity, as do 'title' and 'film title'. Such synonyms need to be grouped together to identify the distinct 'things' that might help identify potential classes. On the other hand, the same word may be used to refer to different things; for example 'number' is used to refer to the unique identifying characteristic of both DVDs and members, as well as occurring in the phrases 'number of films' and 'number of copies available'.

The requirements document for the DVD Library System is in Section 8 of the *Handbook*.

A synonym for a word or phrase is another word or phrase that means the same as the first, and can be used in its place.

Exercise 1

By grouping together nouns and noun phrases that are synonyms, list all the distinct entities that are mentioned in the requirements for the DVD Library System.

Discussion.....

Here is our list:

library (also referred to as 'DVD Library')
 DVD (also referred to as 'copy')
 number of films
 number of copies
 film
 system (also referred to as 'DVD Library System')
 title (also referred to as 'film title')
 member (also referred to as 'library member')
 loan
 day
 identifying number
 membership number
 member's name
 library staff (also referred to as 'staff member')
 number of copies available
 list
 return date
 fact
 date on which it was borrowed

There are a couple of things worth noting about this list:

- ▶ We have used a singular noun or noun phrase to represent each group of synonyms. This is a step towards assigning class names, which are always singular.
- ▶ In the system domain, 'DVD' – a physical video disk – is synonymous with 'copy'. The copies of the films are held on DVDs. (In other contexts, 'copy' might be synonymous with a recording on some other medium, e.g. a videotape or reel of celluloid). The DVDs – or copies – need to be distinguished from the films themselves, which are intangible. There may be many DVDs of a single film in the library.

The solution to Exercise 1 has advanced the process of identifying the classes needed to model the system domain. In other words, it has resulted in a list of candidate classes. For example, the second item in the list, DVD, suggests that all the DVDs in the library might be modelled by means of a DVD class; every individual DVD entity would be modelled by an instance of this class. However, there clearly remains some weeding out to be done; not all the items in the list will give rise to appropriate classes. Unfortunately, there is no mechanical technique for doing this: no set of criteria could adequately deal with the complexities and vagaries of natural language. Your ability to select appropriate classes will improve with experience, but below are some guidelines that should help you to narrow down an initial list.

Guidelines for rejection of candidate classes

Though there are no hard and fast rules, your answers to the following questions should give you a strong indication of whether or not an entity really should be modelled using a class.

Is it a property?

Is it simply a property of some other type of entity in the system domain, or does it contribute to the system domain in its own right? If the former, it is likely to be appropriately modelled by an attribute of a class rather than a class itself.

As you will see below, a good test of whether an entity should be modelled using a class or an attribute is whether it has or does not have properties itself. If it has properties, then it is likely to be appropriately modelled by a class; if not, then it is likely to be appropriately modelled by an attribute.

Some examples of attributes from the DVD Library are:

- title – an attribute of a film;
- identifying number – an attribute of a DVD;
- membership number – an attribute of a member;
- member's name – another attribute of a member;
- number of copies available – an attribute of film (but see further discussion below);
- return date – an attribute of a loan;
- date on which it was borrowed – an attribute of a loan.

What about 'day'? Number of days is also a property of a loan. However, as all loans are of the same duration – three days – there is no need to record this information for each loan. You will see later how this information is taken into account.

Having decided above that 'number of copies available' is to be modelled by an attribute of a film, should 'number of copies' (that is, the total number of copies of a film, whether on loan or not) also be modelled by an attribute of a film? As you will see below, we believe that it is more appropriate to model 'number of copies' in terms of links between films and DVDs.

Does it refer to a behaviour of the system?

Does it name some process or result of a process that the system has to carry out?

In the DVD Library, a 'list' (of the film titles and return dates for each DVD) is something that the system needs to produce as part of its behaviour, rather than a category of entities that is significant for the system domain itself.

Something else that could fit into this category is 'number of copies available'. We have identified DVD (copy) as another candidate class, and the number of copies available is something that the system is required to produce. However, 'number of copies available' can also be viewed as an attribute of a film, which is how we have chosen to model it.

Is it outside the scope of the system?

Is it a word or phrase that is used to help describe the general purpose of the system or what it consists of, but that does not actually refer to significant entities in the system domain?

The 'library' itself falls into this category. The system will operate in the context of the library, but will not need to represent the library per se. The significant entities are those with which the library is concerned – its members, DVDs etc. You could think of the library as being the boundary of the system domain. Had the system been required to administer a group of libraries, then 'library' might have given rise to a class, since it

Although, strictly speaking, it is the category of entity that is modelled by a class, we shall often use the word entity as if it were a category.

would probably be important to know, for example, which copy of a film was held at which library.

Things such as 'user' or, in the case of the DVD library, 'library staff', also fall into this category; although they will use the system, they are outside its boundaries. In other words, the system does not need to refer to them.

Does it refer to an aspect of the user interface?

Does it concern the interaction of a user with the system, rather than something that is part of the system itself? There are no such examples in the DVD Library System, since the requirements document does not provide details of the user interface. A more detailed requirements document might include statements such as 'The librarian enters the unique identifying number of the DVD using a barcode reader.' In this case 'barcode reader' would be rejected as a class, since it refers to an aspect of the user interface.

Does it refer to connections between other significant entities in the system domain?

Such an entity might be more appropriately modelled by an association between two classes rather than a class. For example, it might appear at first sight that an entity 'loan' in the DVD Library System could be represented as a link between a member of the library and a DVD that they borrow. However, the DVD Library System requirements indicate the need to record the date on which a DVD is borrowed, and to provide information on when DVDs are due for return. These dates are properties of a loan, so in this case we will model loans using a class, with the two dates as attributes. However, if the entity 'loan' had not had properties, then it would have been more appropriate to model it as an association. Criteria for deciding whether an event, such as a loan, should be modelled as a class or an association will be discussed later in the unit.

Is it a word or phrase that is used in talking about systems in general, rather than a reference to something in the system domain?

Words such as 'system', 'behaviour' and 'requirement' fall into this category: they do not refer to entities in the system domain.

Is it simply part of a language idiom?

The requirements document may contain words which are just part of general English usage. For example, 'The system records *the fact* that this copy of the film is no longer available for loan ...' could equally well have been written as 'The system records that this copy of the film ...' (i.e. excluding the phrase 'the fact'), and the sentence 'The DVD Library holds DVDs of *a number of* films' would have meant exactly the same had the phrase 'a number of' been omitted.

A number of alternative solutions are possible when categorising and selecting from potential classes. We might, for example, have categorised 'number of films available' and 'list of titles and return dates' as aspects of the user interface (as this is where they would be displayed).

It may sometimes be the case that an entity in one of the above categories *does* need to be represented by a class in the core system, or that a decision is made to introduce a class not represented by a noun or noun phrase in the requirements document, or indeed not mentioned in the requirements at all. Furthermore, the same entity may perform more than one role. For example, if library members were able to use the system to reserve and check in DVDs themselves, 'library member' would be similar to 'library staff' in this context (and so outside the system boundary), as well as being one of the entities being modelled.

Having applied the guidelines to the entities identified in Exercise 1, we are left with the following:

DVD
film
member
loan

All four of these represent significant entities in the system domain, and none of them duplicates any of the others. They therefore suggest categories of entity that can each be modelled by a different class.

Notation for classes

For reasons of clarity it will be useful to distinguish things in the real world, such as the list of categories produced above, from the conceptual classes and objects that will represent them in our model. To make this distinction clear, we will be showing, as above, categories of entity from the system domain in the same font as the rest of the text, and starting with a lower case letter unless at the beginning of a sentence (or, as in the case of 'DVD', where the entity name itself starts with an upper-case letter). However, the corresponding conceptual classes in the model (and their attributes) will be denoted in a different font (e.g. DVD). It will be the same font that will be used for the software classes (and attributes) that you will meet later in the course, but at this stage of the analysis it is important to realise that the classes being dealt with are conceptual classes and not software classes.

Class names are always singular nouns, and each name should be a single word starting with an upper-case letter. The name chosen for a class should give some indication of the type of entity its instances are supposed to represent. Where more than one word is needed to achieve this, we will run the words together, capitalising the first letter of each word (e.g. BarcodeReader).

Applying these notational rules to the list of classes we have decided to include in our model for the DVD Library System, we have DVD, Film, Member and Loan.

We will not expect you to concern yourself with the different fonts when doing assignments or examination questions, but you should adhere to the conventions for naming and capitalisation.

Kinds of class

As a complement to textual analysis, attempts have been made to categorise likely classes by type. Aspects of a system domain that can be modelled appropriately as classes include:

- ▶ tangible entities;
- ▶ roles;
- ▶ events;
- ▶ organisational units;
- ▶ abstract entities.

Tangible entities are the physical things in the world of the system domain: aeroplanes, vehicles, reactors, people and so on, i.e. things than can be seen or touched.

Roles are played by people (and other things) in the system domain: employee, student, lecturer, driver and so on. Roles differ from tangible entities in that a single person (a tangible entity) may have a number of different roles, e.g. parent, teacher, library member.

Events. These include any significant circumstance, episode, interaction, happening or incident, e.g. deliveries, registrations, bookings, enrolments. We will discuss the modelling of events in more detail in Section 5.

Adapted from Schlaer and Mellor (1988), and Coad and Yourdon (1991).

Organisational units. These include any parts of organisational structures to which people or things in the system domain belong – departments, faculties, branches, regions and so on.

Abstract entities. These are things such as plays, novels, pieces of music, theories and political movements. They have an existence but do not correspond to tangible entities. For example, the play Hamlet is an abstract entity, while the books which contain the play are tangible entities.

While consideration of categories such as the above can be useful, you should always remember that the overall purpose is to identify all the classes needed to model significant aspects of the system domain. There will sometimes be classes that do not fit neatly into any of the above categories.

Exercise 2

Consider each of our classes for the DVD Library System in terms of the categories above. For each class, decide whether the entities it models fit into one of the five categories, and if so which one.

Discussion.....

The candidate classes can be categorised as follows:

- DVD – tangible entity;
 - Member – role;
 - Film – abstract entity;
 - Loan – event.
-

Identifying attributes for the classes in the DVD Library System

As a by-product of our class rejection process, we have already identified a number of entities that can appropriately be modelled as attributes. A similar font and naming convention to that for classes will be applied to attributes, except that their names will start with a lower case letter.

Attributes of Film: title, numberAvailable

Attribute of DVD: number

Attributes of Member: number, name

Attributes of Loan: returnDate, issueDate

To check whether there are any others, it is necessary to read through the requirements document to ensure that no information about the properties of classes has been missed. In fact, none of the classes that we have identified has any additional attributes.

For a simple system it is often possible to identify attributes in parallel with classes, particularly when all the references to attributes are nouns. But suppose a requirements document contained something like the following:

Each warehouse contains a number of bar-coded products.

Here, the term 'bar-coded', although it is an adjective and not a noun, points to the need for a barcode attribute in the Product class. Unless the noun 'barcode' appeared elsewhere in the description, this could well have been missed during the class-identification process.

3.3 Identifying classes and attributes for the Hospital System

Identifying the classes for the Hospital System

Now that you have been armed with a number of techniques for identifying appropriate classes for a conceptual model, you can apply them to the requirements document for the Hospital System, which was produced in *Unit 2*.

Exercise 3

The requirements document for the Hospital System is set out in Subsection 4.3 of *Unit 2*. However, you will also find it in Section 3 of the *Handbook*.

Use the textual analysis technique described earlier in this section to identify the nouns and noun phrases in the first part of the document, headed *System domain*, and then eliminate any duplicates or synonyms so as to produce a partial list of entities in the system domain. (To identify all the relevant entities, you would also need to analyse the section of the document entitled *Use cases*, but you are only required to carry out part of the process).

Discussion.....

Figure 4 contains the description with the nouns and noun phrases underlined.

System domain

In the hospital, there are a number of wards, each of which may be empty or have on it one or more patients. There are two types of ward, male wards and female wards. A ward can only have patients of the specified sex on it. Every ward has a fixed capacity, which is the maximum number of patients that can be on it at one time (i.e. the capacity is the number of beds in the ward). Different wards may have different capacities. Each ward has a unique name. The hospital has an Administration department that is responsible for recording information about the hospital's wards and the patients that are on each ward.

The doctors in the hospital are organised into teams, each of which has a unique team code (such as Orthopaedics A, or Paediatrics). Each team is headed by a consultant doctor who is the only consultant doctor in the team; the rest of the team are all junior doctors, at least one of whom must be at grade 1. Each doctor is in exactly one team. The Administration department keeps a record of these teams and the doctors allocated to each team.

Each patient is on a single ward and is under the care of a single team of doctors; the consultant who heads that team is responsible for the patient. A patient may be treated by any number of doctors but they must all be in the team that cares for the patient. A doctor can treat any number of patients.

Figure 4 Identifying nouns and noun phrases

This is not an exact science, so do not worry if your underlining does not exactly match ours.

After eliminating duplicates, we are left with the following list of entities (synonyms are shown in parentheses):

- hospital
- number of wards
- patient
- type of ward
- male ward
- female ward
- ward
- sex
- capacity (maximum number of patients, number of beds)
- name of ward
- Administration department
- information
- doctor
- team
- team code
- Orthopaedics A
- Paediatrics
- grade 1
- consultant doctor (consultant)
- junior doctor
- record of teams and doctors
- care
- number of doctors
- number of patients

Your list may not have been identical to ours, but we would expect you to have included most of the entities above.

Applying the guidelines for rejection to our solution to Exercise 3, the following can be eliminated:

- hospital, Administration department (outside the scope of the system)
- number of wards (language idiom – the requirements document could equally well have referred to ‘wards’ or ‘some wards’)
- type, capacity, name (attributes of ward)
- sex (attribute of patient)
- information (generic term referring to the behaviour of the system)
- team code (attribute of team)
- record of teams and doctors (part of the behaviour required of the system)
- number of doctors, number of patients (language idiom)

You may have noticed that there are three nouns, ‘Orthopaedics A’, ‘Paediatrics’ and ‘grade 1’, which have not been eliminated using the guidelines for rejection, yet which common sense suggests should not be modelled by classes. ‘Orthopaedics A’ and ‘Paediatrics’ are in fact given in the requirements document as example values of ‘team code’, which was identified above as an attribute of team, and ‘grade 1’ is a value of an

attribute, 'grade', of junior doctor. (The grade attribute will be identified in Exercise 4 below).

This leaves, so far, the following list of entities which give rise to candidate classes:

- male ward
- female ward
- ward
- patient
- doctor
- team
- consultant doctor
- junior doctor
- care

Identifying additional classes from the use cases

To finalise the above list, it is necessary to analyse the section of the requirements document dealing with the use cases of the system, and to apply the same guidelines and criteria for rejection. This may identify the need for additional classes, or it may provide information that enables us to eliminate more classes from our existing list.

Having underlined all the nouns and noun phrases in the *Use cases* section of the requirements document, eliminated classes already identified, and applied the rejection guidelines, we believe there are no additional classes. You may want to convince yourself of this by reading carefully through the *Use cases* section.

However, in our view, analysing the *Use cases* section of the requirements document allows us to replace two of the candidate classes by a single class, and reject one further class.

- *Male and female wards.* We initially retained these as potentially giving rise to separate classes because we could not be sure that the two types of ward played a similar role in the system. However, the *Use cases* section of the requirements document makes clear that the two types of ward differ only in the sex of the patients that can be allocated to them. These classes can therefore be rejected in favour of a single class modelling wards with an attribute to denote the sex of the patients on the ward. (Note that in the similar case of junior doctor and consultant doctor, both classes are retained because there are differences in the role that each plays in the system domain.)
- *Care.* The care of a patient, as described in the requirements document, is significant only in respect of which team provides that care; the description identifies no specific functionality or attributes with respect to the care itself. We conclude that (at this point) there is no need for a class modelling care, and that the care of a patient by a team can be modelled as a link between the relevant `Patient` and `Team` objects.

This leaves the following list of conceptual classes:

- Ward
- Patient
- Doctor
- Team
- ConsultantDoctor
- JuniorDoctor

Notice that these classes, now accepted as conceptual classes, are in the special font for representing conceptual classes.

Identifying attributes for the classes in the Hospital System

The next task is to identify the attributes of the conceptual classes listed above. We have already identified some of these as a by-product of the process of identifying the classes.

Exercise 4

By referring to the *System domain* and *Use cases* sections of the requirements document, suggest attributes for each of the classes in the Hospital System.

Discussion.....

For each of our chosen classes, here are our lists of attributes, together with references to the relevant parts of the requirements document.

Ward: name, type, capacity, numberOfFreeBeds (*System domain, use cases A, D, F*)

Patient: name, sex, dateOfBirth, age (*System domain, use cases A, F*)

Doctor: name (*System domain, use case G*)

Team: teamCode (*System domain, use case G*)

JuniorDoctor: name, grade (*use case G*)

ConsultantDoctor: name (*use case G*)

Having identified a complete set of classes and attributes, they should be described formally, accompanied by short comments, as follows. Such **class descriptions** form an important part of the conceptual model.

Class	Ward	A ward in the hospital
	Attributes	
	name	The unique name of the ward
	type	Whether the ward is for male or female (M or F) patients
	capacity	The maximum number of patients that can be on the ward at any one time
	numberOfFreeBeds	The number of free beds on the ward
Class	Patient	A patient in the hospital
	Attributes	
	name	The name of the patient
	sex	The sex (M or F) of the patient
	dateOfBirth	The date of birth of the patient
	age	The age of the patient in years
Class	Team	A team of doctors
	Attributes	
	code	The unique code of the team

Class	Doctor	A doctor at the hospital
	Attributes	
	name	The name of the doctor
Class	JuniorDoctor	A junior doctor at the hospital
	Attributes	
	name	The name of the junior doctor
	grade	The grade (1, 2 or 3) of the junior doctor
Class	ConsultantDoctor	A consultant doctor at the hospital
	Attributes	
	name	The name of the consultant doctor

3.4 Generalisation relationships

We have already discussed links between particular entities in the system domain, and briefly introduced the concept of an *association* between two classes – a relationship whereby the instances of those classes might be linked in a way that is significant for the system domain.

SAQ 3

Suggest a possible association between any two of the classes `Team`, `ConsultantDoctor` and `Patient`, and give it a suitable name.

ANSWER.....

You might have suggested any of the following, possibly with slightly different names:

- ▶ `caresFor` (or `isCaredFor`) between the classes `Team` and `Patient`;
- ▶ `isHeadedBy` (or `heads`) between the classes `Team` and `ConsultantDoctor`;
- ▶ `isResponsibleFor` (or `isResponsibilityOf`) between the classes `ConsultantDoctor` and `Patient`.

Note that associations are highlighted by use of the same font that is used for classes and attributes.

We will discuss associations in detail in Section 4 of this unit. But there is another, quite different, way in which classes can be related. In the context of the Hospital System the classes `JuniorDoctor` and `ConsultantDoctor` are each related to the class `Doctor` by virtue of the fact that all junior doctors and all consultant doctors are also doctors. This is referred to as a **generalisation** relationship between the classes `Doctor` and `JuniorDoctor`, and also between `Doctor` and `ConsultantDoctor`; that is, the class `Doctor` *generalises* each of the classes `JuniorDoctor` and `ConsultantDoctor`.

To put it another way, each of the classes `JuniorDoctor` and `ConsultantDoctor` is a **specialisation** of the class `Doctor`; that is, each of the classes `JuniorDoctor` and `ConsultantDoctor` *specialises* `Doctor`. The `Doctor` class is referred to as the **parent class** or **superclass** of `JuniorDoctor` and `ConsultantDoctor`, with the latter two classes being called **child classes** or **subclasses** of `Doctor`.

You saw earlier in the unit that a conceptual class is represented in a class diagram by a rectangle containing the name of the class and (optionally) the names of its attributes. Figure 5 shows two ways of representing generalisation relationships in a class diagram.

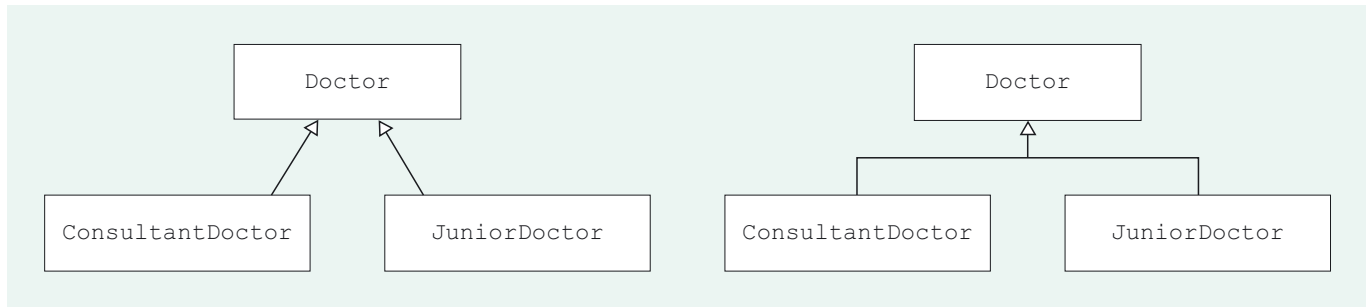


Figure 5 Depicting the generalisation of JuniorDoctor and ConsultantDoctor by Doctor

Either style of diagram is equally acceptable. In your own work you should produce whichever style is easier for you.

Note that the arrowhead points to the more general class. The orientation of the diagram is not significant: there is no need for a child class to appear *beneath* its parent class in the class diagram, although it frequently does so in practice.

The implication of specialisation is that a specialised class has its attributes and associations defined in two ways: those that are common to all the objects of the more general class, and those that are specific to itself. When we identified attributes for the classes in the Hospital System in Exercise 4, we decided that all three of the classes representing doctors would require a name attribute. In view of what we have just said – that a specialised class has all the attributes of its parent class – this attribute can be removed from the JuniorDoctor and ConsultantDoctor classes, provided it is made clear in the model that both JuniorDoctor and ConsultantDoctor specialise Doctor. In object-oriented parlance, the child classes in a generalisation relationship are said to **inherit** properties from their parent class. So JuniorDoctor objects, for example, will have a name attribute inherited from Doctor, as well as a grade attribute, which is specific to the JuniorDoctor class.

You will see later that the class Doctor has an association with the class Patient (representing the treatment by doctors of patients). JuniorDoctor and ConsultantDoctor will also inherit this association. This is appropriate since, for example, the *List Patient's Treatment* use case for the Hospital System specifies that the system must be able to list the names of doctors who have treated a patient. It does not specify a particular kind of doctor: those listed may well include both junior doctors and a consultant doctor. This relationship between doctors and patients really means that there may be links between junior doctors and patients, and between consultant doctors and patients.

It is important to recognise that identifying a generalisation relationship at the conceptual level (i.e. in the conceptual model) does not necessarily mean that the developers are committed to creating a similar inheritance hierarchy at implementation.

In deciding whether a generalisation relationship is appropriate, it is necessary to consider the kind of entity each class is modelling as well as any common properties and relationships. For example, a ward has a name, but a ward is clearly not a kind of doctor! An appropriate generalisation relationship is one in which it is possible to use a real-world entity represented by an instance of the child class in place of a real-world entity represented by an object of its parent class. This property, in which an instance of one class can be substituted for an instance of another, is known as **substitutability**. Thus in the statement 'Each doctor is in exactly one team', 'junior doctor' can be substituted for 'doctor'.

Here are the modified class descriptions for the `Doctor` class and its subclasses. Note the inclusion of comments to indicate the generalisation relationship.

Class	<code>Doctor</code>	A doctor at the hospital Generalises <code>JuniorDoctor</code> and <code>ConsultantDoctor</code>
Attributes		
	<code>name</code>	The name of the doctor
Class	<code>JuniorDoctor</code>	A junior doctor at the hospital Specialises <code>Doctor</code>
Attributes		
	<code>grade</code>	The grade (1, 2 or 3) of the junior doctor
Class	<code>ConsultantDoctor</code>	A consultant doctor at the hospital Specialises <code>Doctor</code>
Attributes		
	None	

3.5 Abstract classes

Exercise 5

Read the following extract from the requirements for the Hospital System:

Each team is headed by a consultant doctor who is the only consultant doctor in the team; the rest of the team are all junior doctors, at least one of whom must be at grade 1. Each doctor is in exactly one team.

What can you infer from this extract about `Doctor` objects?

Discussion.....

Because every doctor is in a team, and the only doctors in a team apart from the consultant are junior doctors, you can infer that each `Doctor` object is in fact an instance of either `ConsultantDoctor` or `JuniorDoctor`. In effect, there are no `Doctor` objects as such.

A class whose 'instances' are all actually instances of one of its subclasses is called an **abstract class**. You may already have met the concept of an abstract software class in your study of object-oriented programming. Java does not allow you to create any instances of abstract classes, since every 'instance of the abstract class' is actually an instance of one of its subclasses. Do not confuse an abstract class with a class representing 'abstract entities', which you met at the end of Subsection 3.2. An abstract class is a class which conceptually has no instances of its own, but only instances of its subclasses. A class representing abstract entities may have instances, but instances which represent intangible entities like novels.

In M256 we denote an abstract class in a class diagram by adding `<<abstract>>` to the box representing the class, as shown in Figure 6. The `<<...>>` notation in the UML is known as a **stereotype**, which is a label giving additional information about some component of a model.

The word 'abstract' is commonly used to refer to something which is intangible or not concrete.

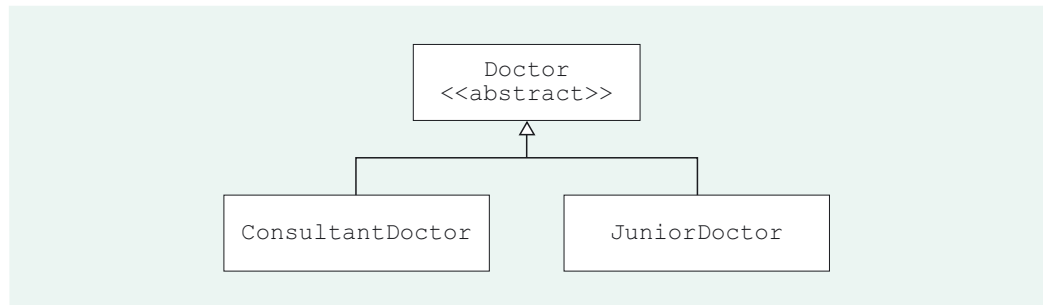


Figure 6 An abstract class with two subclasses

By no means all parent classes are abstract. If a class is not shown as <<abstract>>, it is assumed to be a **concrete class**, i.e. it is intended that the class may have instances that are not instances of any of its subclasses.

Exercise 6

Consider the three following excerpts from requirements documents. In each case decide whether any classes appear to be involved in a generalisation relationship. Where you decide that a generalisation relationship exists:

- ▶ make a sketch of the generalisation relationship, in either of the styles shown in Figure 5. Your sketch should include a name for each class and should indicate which of the classes, if any, are abstract;
 - ▶ suggest a set of attributes for each class involved in the generalisation relationship.
- (a) A department store has a number of departments, each of which is headed by a manager, whose salary depends on their grade. Sales assistants are assigned to a particular department. For security purposes each sales assistant is provided with a personal code, which they need to enter into the till when they make a sale. Department managers do not conduct sales transactions, though they sometimes advise customers on products. All staff, including department managers and sales assistants, have unique employee numbers.
 - (b) A car-hire firm rents out cars and vans. The firm identifies the vehicles in its fleet by their registration numbers. Customers book rental vehicles via the company's website. When booking a car the customer needs to select a type of car, e.g. 'family saloon' or 'luxury model', whereas when booking a van the customer needs to specify the load-carrying capacity, e.g. '800 kg' or '1.5 tonnes'.
 - (c) A university records details of all its lecturers and students, including their postal and email addresses. All lecturers must teach at least one module each year, and may not teach more than four. Students may choose to study up to six modules each year.

Discussion.....

(a) Here is our diagram.

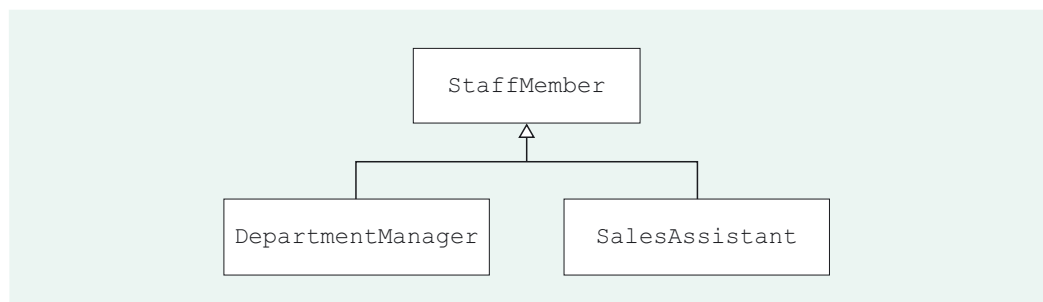


Figure 7 Generalisation relationship representing staff in the department store

This description indicates a generalisation relationship with `StaffMember` as the parent class, and `DepartmentManager` and `SalesAssistant` as its child classes. The final sentence in the extract implies that there are more categories of staff than just department managers and sales assistants. We do not know anything about these categories other than that they have an `employeeNumber` attribute, but it is reasonable to assume that they have some significance in the system domain. From the information provided, they can be modelled by the `StaffMember` class, which should therefore be a concrete class.

Suitable attributes for the three classes would be:

For `StaffMember`: `employeeNumber`

For `DepartmentManager`: `grade`, `salary`

For `SalesAssistant`: `code`

(b) Here is our diagram.

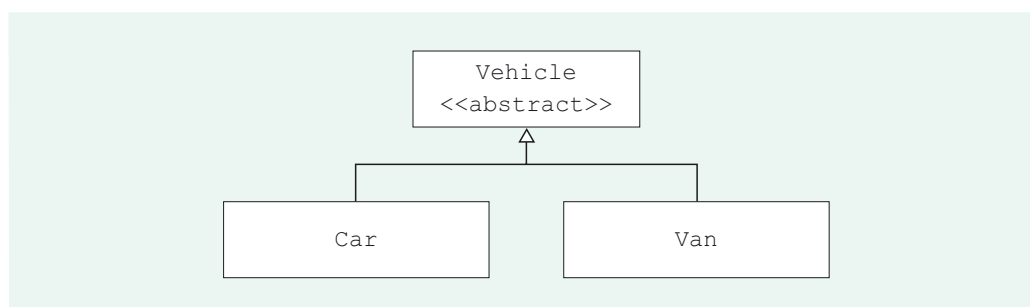


Figure 8 Generalisation relationship representing vehicles in the car-hire firm

This description indicates a generalisation relationship with `Vehicle` as the parent class, and `Car` and `Van` as child classes. The extract implies that all vehicles in the fleet are either cars or vans, so `Vehicle` can be designated as an abstract class.

Suitable attributes are as follows:

For `Vehicle`: `registrationNumber`

For `Car`: `type`

For `Van`: `capacity`

(c) Conceptually, there is no generalisation relationship between `Student` and `Lecturer`; a student is not 'a kind of' lecturer, nor is a lecturer 'a kind of' student. The attributes of lecturers and students, as described in the extract, are identical, and both have a relationship with modules (though this relationship is not the same in each case – lecturers *teach* modules and students *study* them). However, the key point is that, conceptually, one class is not a generalisation of the other.

Students and lecturers do have some things in common, and this might have led you to introduce a parent class for `Student` and `Lecturer` classes, called something like `UniversityMember`. The reason why we consider this to be an inappropriate model at this stage is explained below.

In the Hospital System we noted that both `JuniorDoctor` and `ConsultantDoctor` are subclasses of the abstract class `Doctor`. Since `Doctor` is an abstract class, it will not have any instances, so it might be thought that we could do without a `Doctor` class. The key reason why it should be retained is that it models a significant category of entities in the system domain.

The requirements document has significant things to say about doctors in general, for example:

Each doctor is in exactly one team.

And:

A patient may be treated by any number of doctors but they must all be in the team that cares for the patient. A doctor can treat any number of patients.

As the aim is to model the system domain as closely as possible, it is entirely appropriate that a `Doctor` class should be included. In contrast, the description of aspects of a university in Exercise 6(c) above says nothing about ‘members of the university’, a category which would include students and lecturers. So although students and lecturers have some shared attributes, there is no evidence to indicate that a grouping of these two categories is of significance to the system domain, and therefore no good reason for including a `UniversityMember` class as a parent class of `Student` and `Lecturer`.

As a further example, consider the system for booking badminton and squash courts described in SAQ 2 (at the end of Subsection 2.3). Here is part of the system description:

A computer system is required for booking badminton and squash courts in a sports centre ...

Suppose that classes identified as necessary for modelling the system domain include `SquashCourt`, `BadmintonCourt`, `Booking` and `Member`.

Although only badminton courts and squash courts are explicitly referred to, these two entities are clearly kinds of court, and the description indicates that they have a shared significance in the system domain – the fact that they are bookable. It could therefore be appropriate to include an abstract parent class, `Court`.

3.6 Class or attribute?

In determining suitable classes and attributes for the Hospital System, this section has presented a number of guidelines and techniques as if applying these were a fairly straightforward process:

- ▶ conduct a textual analysis;
- ▶ apply the rejection guidelines;
- ▶ complement this by considering categories of likely classes;
- ▶ identify the attributes;
- ▶ identify any generalisation relationships.

In practice, arriving at a set of appropriate classes and attributes is rarely so straightforward. Rather than working with a short, relatively simple description, you may find that a requirements document describes a system domain with much greater complexity of detail and meaning. One issue that arises time and time again is whether something is an entity or the *property* of an entity. Should it be modelled as a class or an attribute?

This difficulty can be illustrated with reference to a couple of examples.

Suppose that during the requirements specification phase you had asked the Chief Physician what sort of detail is needed about doctors, and that she had replied:

As well as a name, a doctor has a title; also, a record is kept of the institution where he or she qualified.

It seems fairly obvious that title is just another attribute of `Doctor`, but how would you model the reference to institution? Would you introduce an `Institution` class or would it be sufficient, for example, to add an `institutionName` attribute to the `Doctor` class? This depends on what sort of information about institutions needs to be recorded. You would need to question the Chief Physician further. Suppose that in response to further questioning she added:

I need to know how many of my junior doctors qualified at a particular institution, and the name of the Chief Physician at that institution.

This indicates that `Institution` is significant in its own right – it has properties – and will need to be included as a class in the model. On the other hand, her reply might have been:

Just the name of the institution will do. That will enable me to research any further details.

In this case an `institutionName` attribute in `Doctor` would be all that was needed.

Now let's revisit the vehicle hire example in Exercise 6(b). The description contains the following statement:

When booking a car the customer needs to select a type of car, e.g. 'family saloon' or 'luxury model' ...

From this information alone, it was reasonable to model `type` as an attribute of `Car`, but what you know about car hire might lead you to question whether the type of car might have a significance in its own right. Wouldn't the system need to record the properties of each type of car, such as its number of seats or hire cost? If so, then `TypeOfCar` might be a valid conceptual class. As an analyst you would need to question the client further about this.

One of the strengths of an object-oriented development approach such as the one adopted in this course is that the activity of building a conceptual model forces the developer to examine the meaning of statements about the system domain. This iterative questioning is also indicative of a good analytical approach in general – an active exploration of meaning rather than a passive acceptance of 'requirements'.

Having identified an initial set of classes and attributes for the Hospital System (and which of these classes might be generalisations of others, and/or abstract), it is now possible to move on and consider the associations between the classes. However, it is important to bear in mind that the process of developing a conceptual model is an iterative one, and that this set of conceptual classes is still provisional. It may be that the analysis of the associations between classes will require them to be revised.

4 Associations

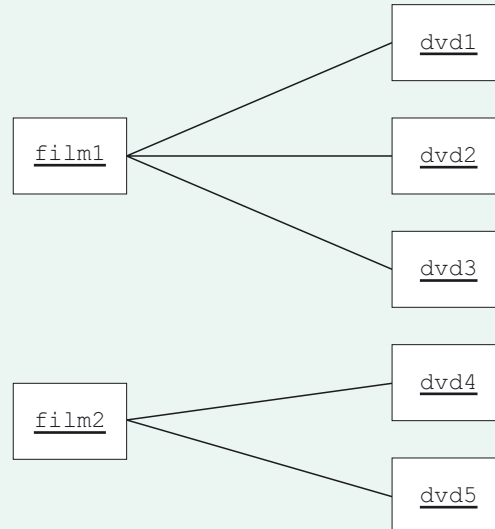
In the previous section you learnt how to identify conceptual classes and their attributes from a requirements document.

In this section you will:

- ▶ review what is meant by an association;
- ▶ learn how to identify associations.

4.1 What is an association?

Just as a class can be thought of as a set of objects that have common attributes and behaviour, so an *association* between two classes is a set of links that have a common meaning. You have seen that a *link* is a connection between a pair of objects – one from each class – that is significant for the system domain. For example, in the DVD Library System, each `Film` object may be linked to a number of `DVD` objects, with each link between a single `Film` object and a single `DVD` object representing a connection between a film and a DVD. You will recall that the situation at a particular point in time can be illustrated by an object diagram, such as the one in Figure 9.



The inclusion of attribute names and values in object diagrams is optional, and we will not normally include them in this unit.

Figure 9 Object diagram illustrating links between `Film` and `DVD` objects

The links in Figure 9 each model the fact that the particular `DVD` object at one end of the link represents a DVD which 'is a copy of' the film represented by the `Film` object at the other end. So these links are said to be instances of an association that we might call `isCopyOf`, which exists between the classes `Film` and `DVD`.

As with classes and attributes, the name given to an association should be a single word and should give an indication of the meaning of the association. The usual convention is for association names to start with a lower-case letter. Note that the name of an association does not imply anything about its direction. This means that naming the

above association `isCopiedBy` (meaning a film is copied by a DVD) would have been just as valid as naming it `isCopyOf` (meaning a DVD is a copy of a film).

4.2 Identifying associations

In practice, the identification of associations often proceeds in parallel with the identification of classes. When you are considering a possible class, it is also natural to think about the connections that the entities it represents might have with other entities in the system domain.

However, as with classes, textual analysis can also help to identify associations. Just as nouns and noun phrases can suggest candidate classes, so the grammatical structures that connect these nouns and noun phrases in the text of a requirements document can suggest likely associations. As an example, here again is the requirements document for the DVD Library System. We have started our search for suitable associations by first highlighting the nouns and noun phrases that correspond to the conceptual classes which have already been identified (by showing them in bold italic font), and we have then underlined any grammatical structures that connect these classes.

System domain

The DVD Library holds ***DVDs*** of a number of ***films***. It may have any number of ***copies*** of each ***film***. The system should, for each ***film***, hold the film's title; no two ***films*** have the same title.

A ***library member*** can borrow any ***DVD*** of a ***film*** that is not currently on ***loan***. However, no ***member*** can have more than six ***DVDs*** on ***loan*** at once. A borrowed ***DVD*** must be returned within three days. When a ***library member*** returns a ***DVD*** it immediately becomes available for ***loan*** to other ***members***.

Each ***DVD*** has a unique identifying number.

Each ***member*** of the library has a unique membership number; the system also holds each member's name.

Use cases

The DVD Library System will provide support to the library by enabling the library staff to do the following:

- A Get Number Available.** The staff member identifies the ***film***. The system displays the number of ***copies*** available.
- B List Films.** The staff member identifies the ***member***. The system displays a list of the film titles and return dates for each ***DVD*** that the ***member*** has on ***loan***.
- C Borrow DVD.** The staff member identifies the ***DVD*** and the ***member*** who is borrowing ***it***. The system records the fact that this ***copy*** of the ***film*** is no longer available for ***loan***, that ***it*** is on ***loan*** to the ***member*** identified, and the date on which ***it*** was borrowed.
- D Return DVD.** The staff member identifies the ***DVD***. The system records the fact that this ***copy*** of the ***film*** is now available for ***loan*** and that ***it*** is no longer on ***loan*** to the ***member*** who returned ***it***.

Concentrating on the nouns which represent classes allows us to disregard relationships that involve other aspects of the system, such as attributes, general behaviour and user interface, or entities outside the scope of the system. Here are some examples of the sorts of thing that can be ruled out:

- ▶ relationships involving classes and their attributes, e.g. 'films have title';
- ▶ relationships describing the behaviour of the system, e.g. 'system displays number of copies';
- ▶ Relationships involving entities outside the scope of the system, e.g. 'library holds DVDs', 'staff member identifies member'.

Remembering that 'copy' is a synonym for 'DVD', underlined phrases such as

DVDs of a number of **films**

copies of each **film**

suggest an association between DVD and Film, representing the fact that a DVD is a copy of a film.

The situation involving members, DVDs and loans is a little more complicated. Here are some of the relevant phrases.

library member can borrow ... **DVD**

library member returns ... **DVD**

copy ... is on loan to the **member**

These might initially appear to indicate an association between Member and DVD, as in the following class diagram.

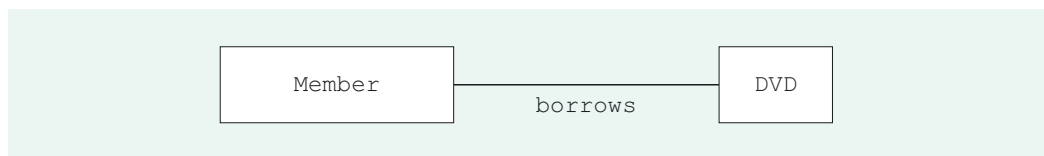


Figure 10 Possible association between Member and DVD

However, the dates on which a loan (or borrowing) is issued and is due for return also need to be modelled, and in M256 associations do not have attributes. This is why we decided to introduce a Loan class. Returning to the phrases above Figure 10, you can see that both members and DVDs (copies) are associated with loans: a loan is to a member and consists of a DVD. So we need two additional associations, giving us three in all:

- 1 isCopyOf between Film and DVD
- 2 isTo between Member and Loan
- 3 consistsOf between Loan and DVD

These associations are illustrated in the class diagram in Figure 11.

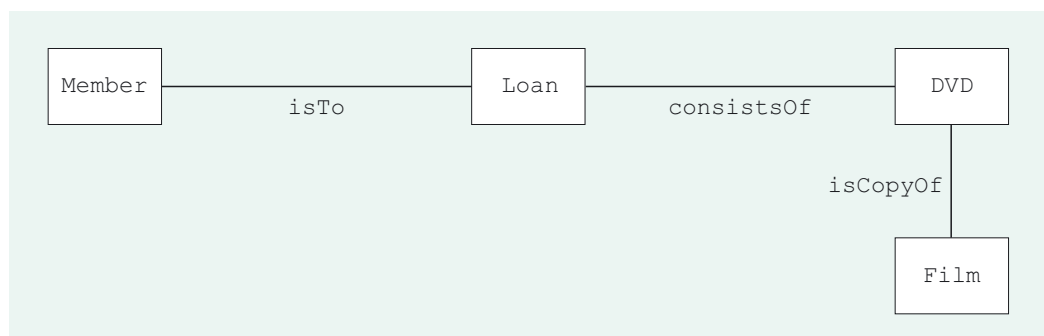


Figure 11 Preliminary class diagram for the DVD Library System

The phrases linking entities which correspond to the classes often do more than just suggest the existence of an association; for example, phrases like 'no member can have more than six DVDs on loan at once' give us information about the number of links that are possible between an object of one class and objects of an associated class at any given time. We will return to this in the next subsection.

The next exercise asks you to apply your new skills of identifying associations to the Hospital System.

Exercise 7

Conduct a textual analysis of the requirements document for the Hospital System and draw up a list of possible associations. Remember that you are only looking for likely associations that involve the conceptual classes that have been identified: `Ward`, `Patient`, `Team`, `Doctor`, `JuniorDoctor` and `ConsultantDoctor`. (You might want to refer back to SAQ 3 in Subsection 3.4, where several of the required associations have already been identified).

The requirements document for the Hospital System is in Section 3 of the *Handbook*.

Discussion.....

Our list of associations is as follows. In each case we have added, in parentheses, a quote from the requirements document which supports the need for this association.

- ▶ `hasOn` between `Ward` and `Patient` ('... wards, each of which may ... have on it one or more patients')
- ▶ `contains` between `Team` and `Doctor` ('doctors ... are organised into teams')
- ▶ `isHeadedBy` between `Team` and `ConsultantDoctor` ('Each team is headed by a consultant doctor')
- ▶ `caresFor` between `Team` and `Patient` ('patient... is under the care of ... team')
- ▶ `isResponsibleFor` between `ConsultantDoctor` and `Patient` ('consultant ... is responsible for ... patient')
- ▶ `hasTreated` between `Doctor` and `Patient` ('A patient may be treated by ... doctors')

The accompanying phrases indicate that all these associations appear to represent significant connections between entities in the system domain. But as with other aspects of the analysis process, identifying associations is not an exact science, so do not worry if your list was not the same as ours. At the very least, some of the names you have chosen will probably differ (though they should still be meaningful).

It is now possible to produce a class diagram (Figure 12) showing the set of classes and associations identified for modelling the Hospital System requirements. The diagram is not yet complete, as it still requires some additional items of information.

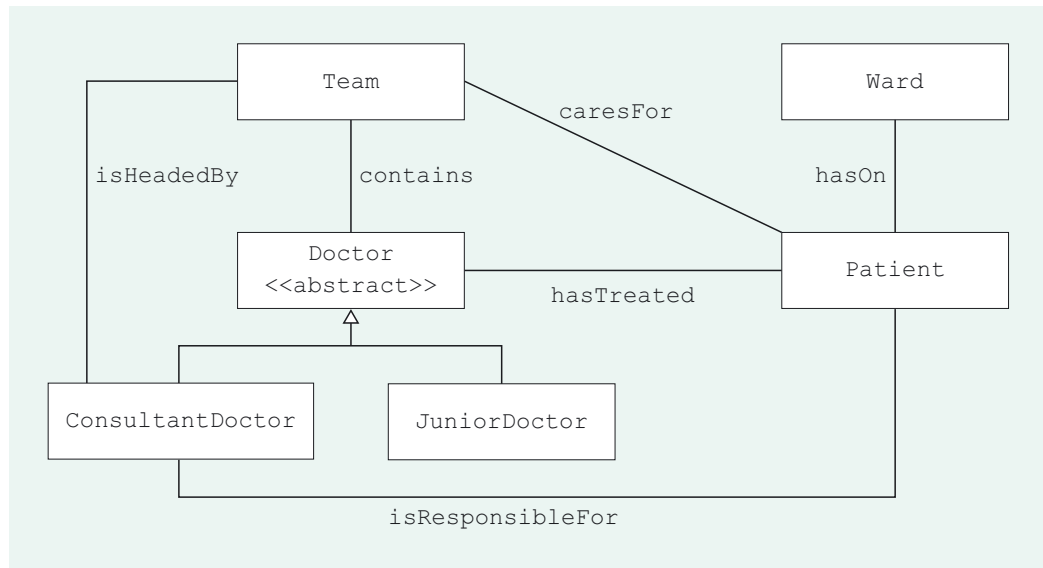


Figure 12 An incomplete class diagram for the Hospital System

Associations and generalisation relationships

Before moving on, it is worth considering briefly the implications of generalisation relationships for associations.

As with attributes, any associations involving a parent class will be inherited by its child class(es). Figure 12 shows two associations involving **Doctor**: `hasTreated` connecting it to **Patient**, and `contains` connecting it to **Team**. The meaning of these associations is that *any* doctor (whether a junior doctor or a consultant doctor) *may* be linked to a team or to patients or both.

Notice that by including the **Doctor** class the structure is simplified. If **Doctor** were omitted, `hasTreated` would need to be replaced by two associations, one between **Patient** and **JuniorDoctor**, and the other between **Patient** and **ConsultantDoctor**; there would be similar ‘duplication’ of the `contains` association.

Because **ConsultantDoctor** inherits an association with **Team** from its parent class, **Doctor**, there are two different associations between **Team** and **ConsultantDoctor**: `isHeadedBy` and `contains`. This is fine, but it does have some implications for the consistency of the model, in that a team must contain the consultant doctor that heads it. We will return to this later in the unit.

4.3 Multiplicity

When discussing the associations for the DVD Library System, we noted that a requirements document frequently provides information about the number of links that are possible between objects of particular classes. For example, the requirements stated that ‘no member can have more than six DVDs on loan at once’. For the sake of simplicity we will assume that a loan involves a single DVD. Therefore, in terms of the model, this means that no **Member** object may have links to more than six **Loan** objects at any one time (though it may have fewer such links).

To illustrate this, the object diagram in Figure 13 represents a situation where there are three members of the library. At the point in time depicted in the diagram, `member1` has three DVDs on loan, `member2` has no current loans, and `member3` has one DVD on loan.

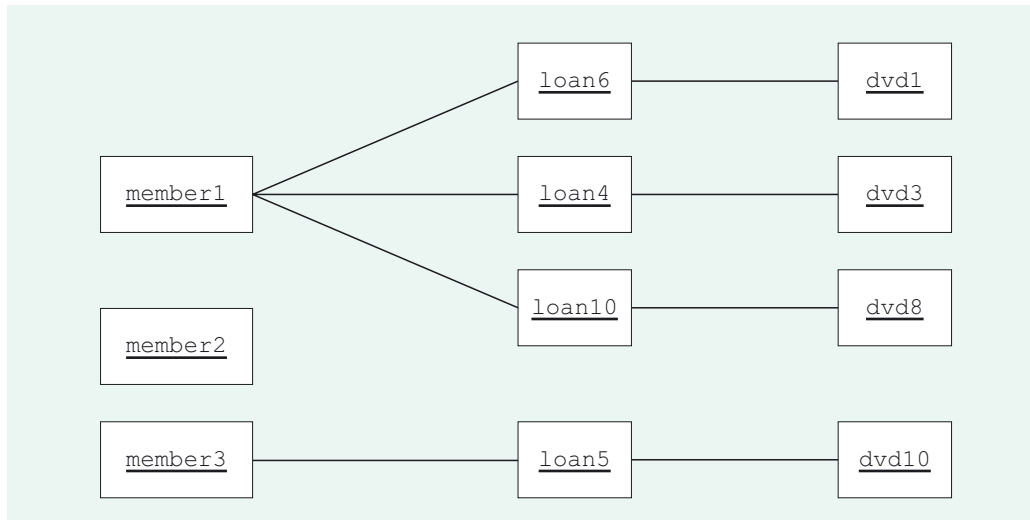


Figure 13 Object diagram showing *isTo* and *isOf* links

The number of instances of a class *B* that can be linked by a particular association to a *single* instance of the class *A* at the other end of the association is known as the **multiplicity** of the association at the *B* end. For example, suppose that each student on a particular course must have a single tutor, and that each tutor must be assigned to 20 students. The multiplicities of a *tutors* association between the classes *Tutor* and *Student* would be shown as in Figure 14.



Figure 14 Multiplicities for a *tutors* association between *Tutor* and *Student*

Figure 14 tells us that each *Student* object must be linked to exactly one *Tutor* object and that each *Tutor* object must be linked to 20 *Student* objects.

The multiplicity may be a single figure or a range. For example, a member of the DVD Library may, at any point in time, have any number of loans between 0 and 6. That is, the multiplicity of the *isTo* association at the *Loan* end is 'zero to six' and would be indicated on the class diagram by writing *0..6* on the *isTo* association next to the *Loan* class. Although a member may have a number of loans at a particular time, each loan will be linked to only a single member, so the multiplicity of *isTo* at the *Member* end is 1 (sometimes expressed as 'exactly one'). Figure 15 shows the class diagram for the DVD Library System with the multiplicities for *isTo* added.

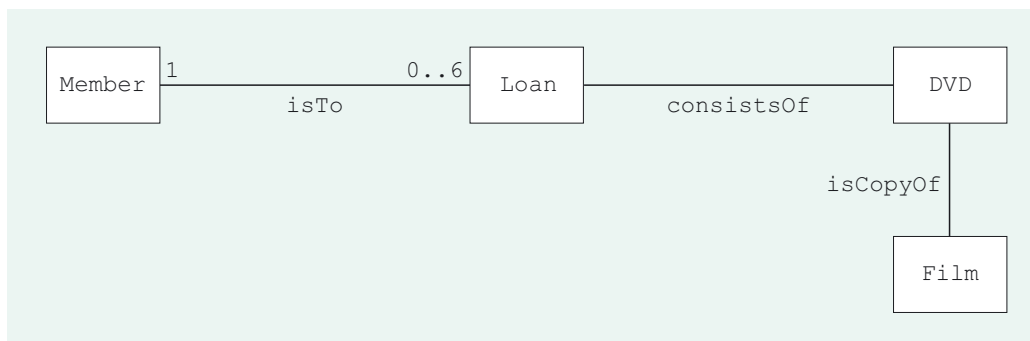


Figure 15 Class diagram for the DVD Library System showing multiplicities for the *isTo* association

Now consider the `consistsOf` association. A particular `Loan` object will always be linked to a single `DVD` object. The multiplicity of `consistsOf` is therefore 1 at the `DVD` end. However, a `DVD` may or may not be on loan, so a particular `DVD` object is linked to zero or one `Loan` objects and the multiplicity of `consistsOf` at the `Loan` end is therefore 0..1 (described as 'zero or one').

SAQ 4

What is the multiplicity of the `isCopyOf` association at the `Film` end?

ANSWER.....

A particular `DVD` is a copy of a single film, so the multiplicity at the `Film` end of `isCopyOf` is 1.

What about the `DVD` end of `isCopyOf`? The requirements state that the library '...may have any number of copies of each film'. In other words, there may be no copies at all, but there is no upper limit. Such a multiplicity is expressed as 0..* (described as 'zero or more'), where * indicates that there are no upper restrictions on the number. Figure 16 shows the class diagram for the system with all the multiplicities added.

In this context, the * is usually referred to as 'star'.

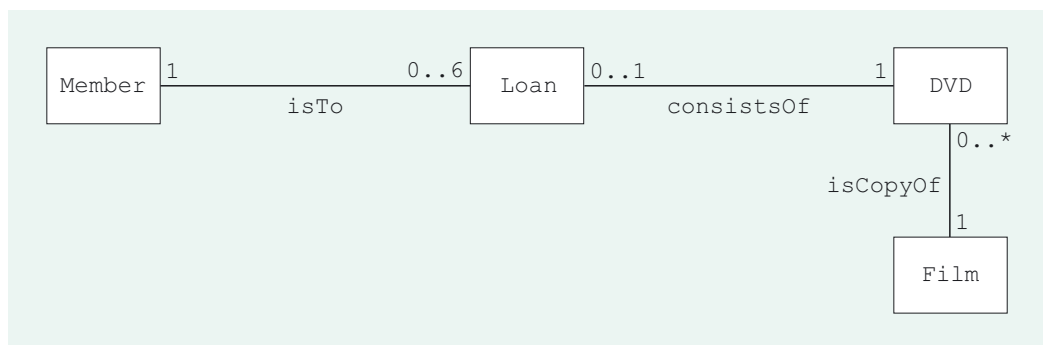


Figure 16 Class diagram for the DVD Library System, showing multiplicities for all associations

Multiplicity ranges do not need to start with 0. A statement such as 'Members of a book club must choose at least two books from the titles on offer each quarter, but may purchase up to six books at the offer price' would give rise to a multiplicity of 2..6 (described as 'two to six') at the `Book` end of a `purchases` association between the classes `Member` and `Book`. Multiplicities are always written with the smaller number in the range first.

SAQ 5

Use Figure 17 (below) to answer the following questions.

- How many courses can a student take?
- How many tutors teach each course?
- How many students can enrol for a course?
- How many courses can each tutor teach?

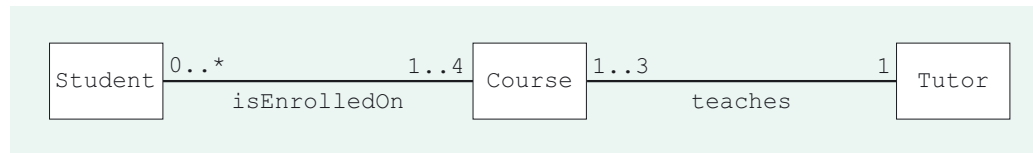


Figure 17 Class diagram representing students, courses and tutors

ANSWER.....

- (a) Between 1 and 4, inclusive (i.e. 1, 2, 3 or 4).
- (b) Exactly 1.
- (c) Zero or more (i.e. any number).
- (d) Between 1 and 3, inclusive (i.e. 1, 2 or 3).

The multiplicities of an association answer important questions about the relationship between the two classes involved. The answers to these questions will play a significant part in later phases of the software development process. Questions which are of particular relevance in this respect are:

- Must an instance of one class be linked to at least one instance of the class at the other end of the association, or is it possible that, at a certain point in time, no such links exist? For example, every `Loan` object in the DVD Library System must be linked to a `DVD` object (a multiplicity of 1), whereas a `DVD` object may or may not be linked to any `Loan` object at a particular point in time (a multiplicity of 0..1).
- May an instance of one class be linked to more than one instance of the class at the other end of the association? For example, a `DVD` object is always linked to a single `Film` object (a multiplicity of 1) at any one time, whereas a `Film` object may be linked to 0, 1 or more `DVD` objects (a multiplicity of 0..*).
- Is there an upper limit to the number of possible links that may exist from an object of one class to objects of the class at the other end of the association? For example, a `Member` object may have at most six links to `Loan` objects.

SAQ 6

Using Figure 17 for reference, describe in words the multiplicities at each end of the `isEnrolledOn` and `teaches` associations.

ANSWER.....

`isEnrolledOn` has a multiplicity of *zero or more* at the `Student` end and *one to four* at the `Course` end.

`teaches` has a multiplicity of *one to three* at the `Course` end and *one (or exactly one)* at the `Tutor` end.

Multiplicities for the Hospital System

Consider first the association `hasTreated` between `Doctor` and `Patient`. Here are some extracts from the requirements document:

A patient may be treated by any number of doctors ...

A doctor can treat any number of patients.

The first extract indicates that there is no upper limit on the number of doctors who have treated a patient. It is also possible that a patient has not been treated by any doctors; they may only just have been admitted to the hospital, for example. So the multiplicity of `hasTreated` at the `Doctor` end is 0..*.

The second extract states that a doctor can treat any number of patients. Allowing for the situation where a doctor has not yet treated any patients, the multiplicity at the *Patient* end is also $0..*$.

Now consider the association *hasOn* between *Ward* and *Patient*. Here are some extracts from the requirements document:

In the hospital, there are a number of wards, each of which may be empty or have on it one or more patients.

Each patient is on a single ward ...

The first extract states that a single *Ward* object may be linked to 0, 1 or more *Patient* objects, indicating a multiplicity of $0..*$ at the *Patient* end of the association. The second says that each patient is on one ward. Thus each *Patient* object is linked to exactly one *Ward* object. The multiplicity of *hasOn* at the *Ward* end is therefore 1.

The class diagram in Figure 18 shows the multiplicities for the *hasTreated* and *hasOn* associations.

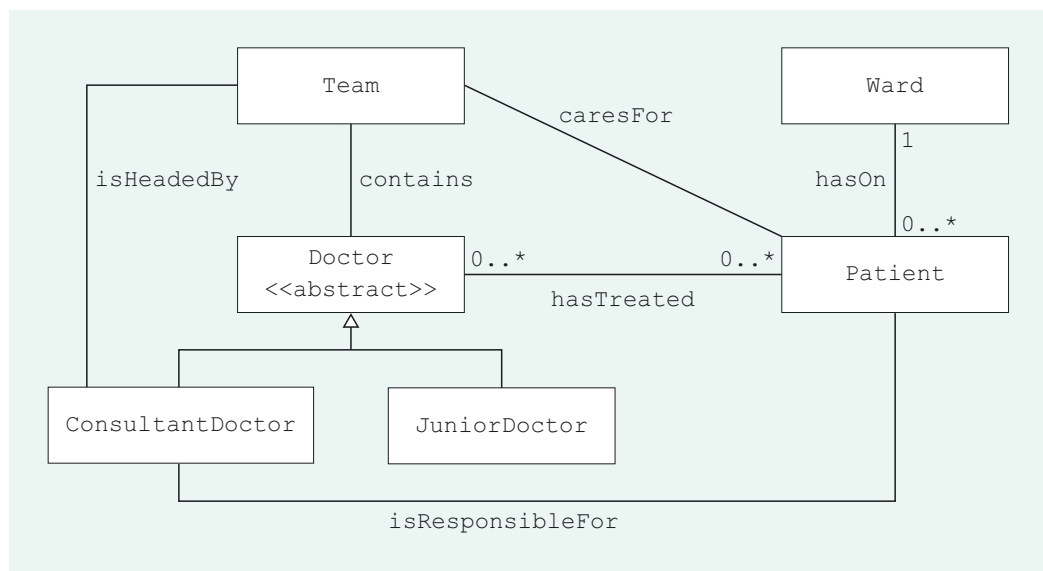


Figure 18 Multiplicities for the *hasOn* and *hasTreated* associations

You may have wondered why the multiplicities for *hasOn* do not take account of the statement 'Every ward has a fixed capacity, which is the maximum number of patients that can be on it at any one time ...' so that in practice no ward can have the unlimited number of patients indicated by a multiplicity of $0..*$. This is because an association represents *all* the links that are its instances. Different wards have different capacities, so it would not be possible for the multiplicity at the *Patient* end of the *hasOn* association to specify an upper limit that would apply to all *Ward* objects. If the requirements had stated, for example, that all wards have a capacity of 15, we could have dispensed with the *capacity* attribute in *Ward*, and modelled the limit by a multiplicity of $0..15$ at the *Patient* end of *hasOn*.

In general, if you do not know the upper limit for a multiplicity, then the best you can do is assign an upper limit of $*$. In practice this is likely to indicate an unknown – and possibly variable – upper limit rather than an infinite one.

Exercise 8

Make a sketch of the diagram in Figure 18, and add the multiplicities for the associations *isHeadedBy*, *contains*, *caresFor* and *isResponsibleFor*. (You will need to refer to the requirements document in Section 3 of the *Handbook*).

Discussion.....

Figure 19 shows our class diagram.

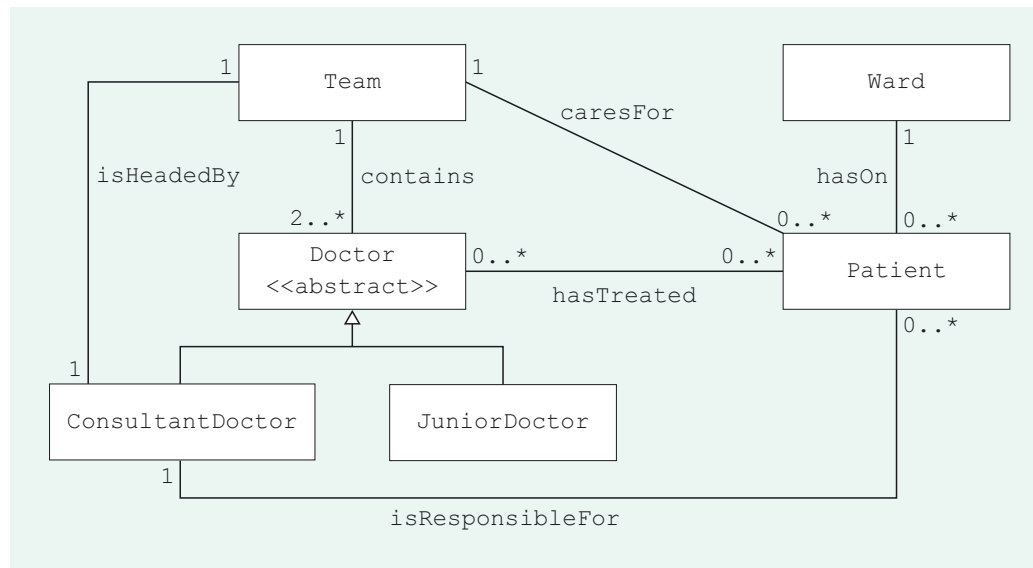


Figure 19 Class diagram for the Hospital System, showing multiplicities

Here is a brief justification of our solution, including relevant extracts from the requirements document:

isHeadedBy:

Each team is headed by a consultant doctor who is ... in the team.

Each doctor is in exactly one team.

Remember that consultant doctors are a kind of doctor, so that it is possible to deduce from the second extract that 'Each consultant doctor is in exactly one team.' Because the first extract specifies that a team must be headed by a consultant doctor who is in that team, it can also be deduced that each consultant doctor can only head one team.

contains:

Each team is headed by a consultant doctor ... in the team.

The rest of the team are all junior doctors, at least one of whom must be at grade 1.

Each doctor is in exactly one team.

The first two extracts indicates that each team must contain at least two doctors – the consultant doctor that heads it, and at least one junior doctor at grade 1, so at least two in all.

The third extract provides the multiplicity at the **Team** end.

caresFor:

Each patient is ... under the care of a single team of doctors

This tells us that each **Patient** object will be linked to exactly one **Team** object. There is no information on the number of patients that can be cared for by a team, so we use * for the upper limit at the **Patient** end. Since there is no information to the contrary, we assume that it is also possible that there are times when a team has no patients under its care, for example when a team is newly constituted, or when all the patients under its care have been discharged.

isResponsibleFor:

... the consultant who heads that team is responsible for the patient.

This is similar to the `caresFor` association. Each patient is the responsibility of a single consultant doctor, and each consultant doctor may have responsibility for zero, one or more patients.

Exercise 9

doctor3 may be a JuniorDoctor object, or it may be a ConsultantDoctor object. The actual class is irrelevant here: any kind of doctor may be involved in treating a patient.

Suppose that a particular patient, represented as the object called `patient5`, has been treated by a particular doctor, `doctor3`, on three different occasions. In an object diagram modelling the system domain at some point after these three treatments have taken place, how many links would be needed between `patient5` and `doctor3`?

Discussion.....

There would be only one link. Although a single `Doctor` object can be linked to zero or more `Patient` objects, there can, for a particular association, only ever be a single link between two particular objects. As far as the conceptual model is concerned, either a particular doctor has treated a particular patient or not. Multiple treatments are not recorded. (We will return to this in the next section.)

A multiplicity specifies the number of *different* instances of a class that can be linked to a single instance of an associated class, not the number of links between particular instances of two classes. However, it *is* possible to have two or more *associations* between two classes, as in the case of `contains` and `isHeadedBy` between `ConsultantDoctor` and `Team`. This might result in there being two links between two objects.

In this section you have learnt various techniques for identifying associations between classes, which represent the links between the objects of these classes. You also learnt about the important role of multiplicities, and how to identify and specify them.

5

Modelling events

In the discussion of classes earlier in the unit, we gave events as one category of ‘thing’ that could be modelled by classes. You saw an example of this in the DVD Library System, where the loan of a DVD by a library member was modelled as a `Loan` class connected to both the `Member` and the `DVD` classes by associations. You have also seen examples where an event is modelled by an association. For example, we chose to model the treatment of a patient by a doctor in the Hospital System as a `hasTreated` association between the `Patient` and `Doctor` classes.

In this section, you will discover some criteria for deciding whether an event should be modelled as a class or as an association.

5.1 Criteria for modelling events

SAQ 7

Why did we choose to model the loan of a DVD as a class, but the treatment of a patient as an association?

ANSWER.....

In the case of the loan event, there were requirements for additional information to be associated with each loan: the date on which the loan was made, and the return date. By modelling ‘loan’ as a class, we were able to model these additional requirements by including appropriate attributes.

In the case of the treatment of a patient, there was no requirement to record or provide any information specific to a treatment, other than the patient and doctor involved. If details of each treatment had been needed, then a `Treatment` class would have been a better choice.

The object diagrams in Figure 20, below, illustrate the examples that were the subject of SAQ 7.

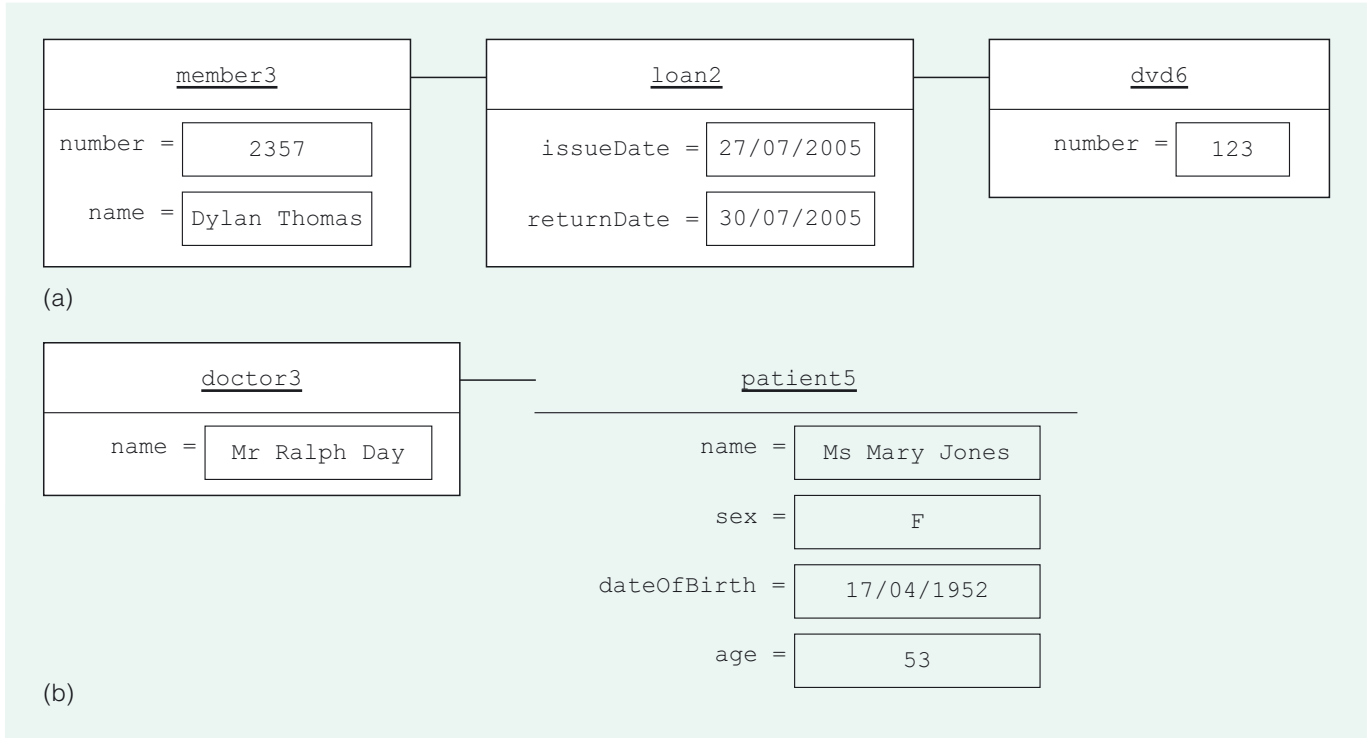


Figure 20 Object diagrams representing loans and treatments

Figure 20(a) shows that `dvd6` is on loan to `member3`. If, at some future date, `member3` borrowed `dvd6` again, the loan involving these two objects would be recorded by a different `Loan` object, say `loan17`.

Figure 20(b) shows only that `doctor3` has treated `patient5`. We know nothing about that treatment, or indeed whether the doctor has treated the patient more than once. In the solution to Exercise 9 in the previous section, we said that it was not possible, with our chosen model, to record multiple treatments of a particular patient, say `patient5`, by a particular doctor, say `doctor3`. If we needed to do this, we would have chosen to model the treatment of a patient by a doctor as a `Treatment` class, each object of which would be linked to a `Patient` object and a `Doctor` object. A class diagram and an object diagram for this possibility, showing the treatments of `patient5` by `doctor3`, are given in Figure 21.

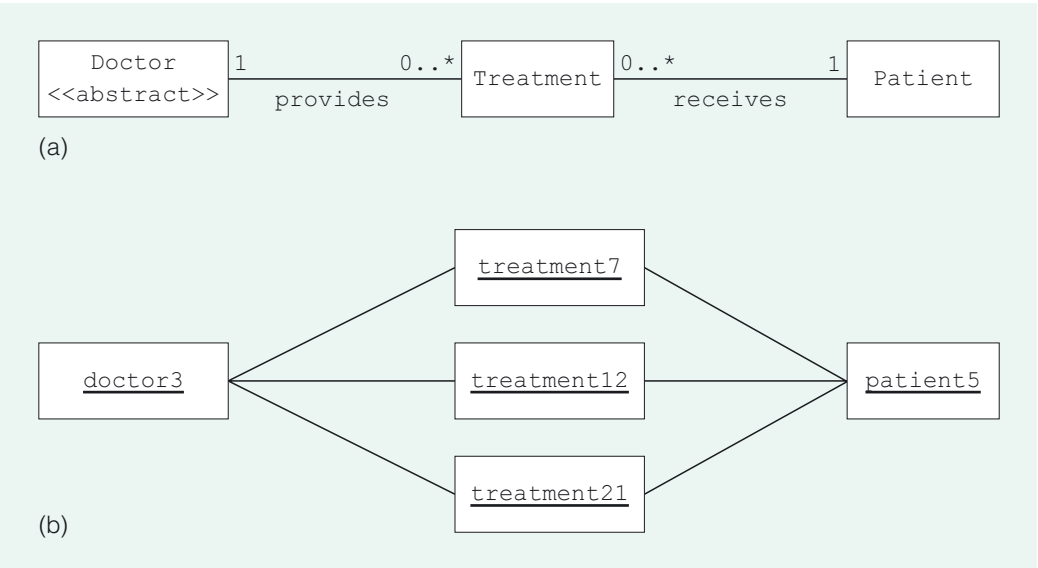


Figure 21 Modelling the treatments of a patient by a doctor with a `Treatment` class

The class diagram (Figure 21(a)) shows that a doctor may provide any number of treatments, and that a patient may receive any number of treatments, but that each treatment involves exactly one doctor and one patient. The `Treatment` class might have attributes to record information such as the date and nature of the treatment, or it might not. The object diagram (Figure 21(b)) shows that this model allows for multiple `Treatment` objects, each involving `doctor3` and `patient5`.

The two example systems which form the basis of the next two exercises demonstrate circumstances in which events that do not have properties of their own are nevertheless modelled as classes. But before discussing these circumstances, we would like you to familiarise yourselves with the requirements of these two systems by completing their class diagrams. In completing the diagrams you will also reinforce your understanding of some of the ideas introduced in the unit so far.

Exercise 10

Here is an excerpt from a requirements document for a Goat Breeding System.

A farm needs to record information on the breeding of its goats at the end of the season. The goats are organised into herds, with each goat belonging to a single herd, and a herd containing many goats. There are three sorts of goat: billy-goats (males), nanny-goats (females) and kids (which, since they do not mate in the current season, are not considered to be either billy-goats or nanny-goats). The farm needs to have a record of the mating behaviour of each nanny-goat and billy-goat during a particular season, and of the offspring which result. A billy-goat mates one or more times during the season, but only with nanny-goats that are in its own herd. A nanny-goat mates exactly once per season. A mating may produce several offspring (kids) or none. All kids remain in the same herd as their parents.

The following classes have been identified as appropriate: `NannyGoat`, `BillyGoat`, `Herd`, `Goat`, `Kid` and `Mating`. Notice that the class `Mating` represents an event involving a nanny-goat, a billy-goat and (possibly) several kids.

Use the above description to identify likely generalisation relationships, associations and multiplicities and draw the class diagram.

Discussion.....

Our class diagram is shown in Figure 22.

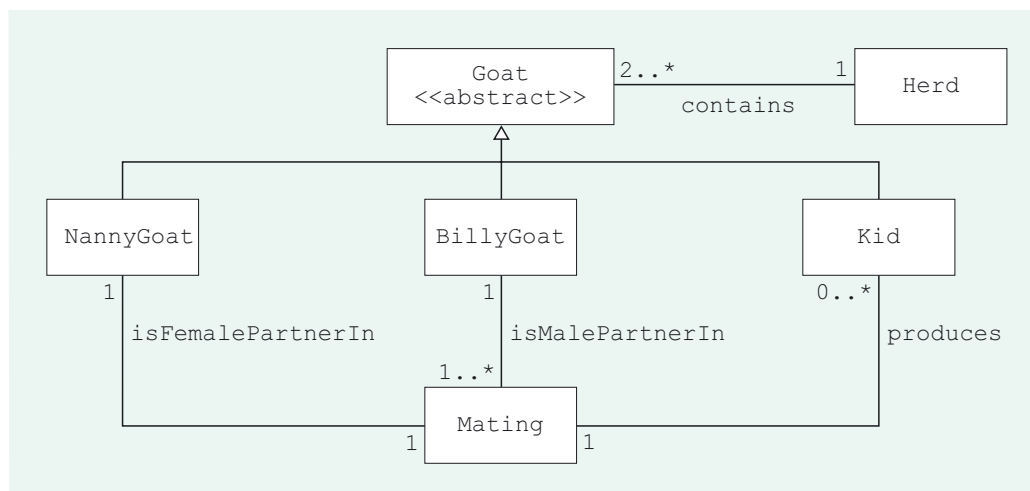


Figure 22 Class diagram for the Goat Breeding System

You have almost certainly come up with different names for the associations. Incidentally, it's worth noting that there is no reason why you should not use the same name for two different associations – for example, you could have given the association between NannyGoat and Mating the same name as the association between BillyGoat and Mating, say *isPartnerIn*. We have avoided the use of duplicate association names in M256, as this would make our models more difficult to discuss.

We have taken the view that a herd must contain at least two goats to exist.

Note that the *Goat* class is abstract, as there are no goats per se, only billy-goats, nanny-goats and kids.

Mating is an example of an event that is modelled by a class. We will return to this shortly.

Exercise 11

Here is an extract from the requirements for a system used in organising an Open University day school, referred to as the Day School System.

A region of the Open University needs to maintain information about a day school for M256. The day school consists of a number of sessions. Two types of participant, tutors and students, are involved in these sessions. The system must record the name and email address of each participant. Each session is taken by a single tutor, and each tutor must take at least one session, though some of the tutors will take a number of sessions. Students need to register for each session that they wish to attend. A maximum of 20 students can be catered for in each session. Once this limit is reached, no further students may register for that session. Only students who register for at least one session are recorded by the system. At each session a different booklet of exercises is used. Each booklet is written by a number of the tutors taking part in the day school, one of whom must be the tutor who is taking the session that uses the booklet.

The following classes have been identified as appropriate: *Session*, *Participant*, *Tutor*, *Student* and *Booklet*.

Use the above description to identify likely generalisation relationships, associations and multiplicities and draw the class diagram.

Discussion.....

Our class diagram is shown in Figure 23.

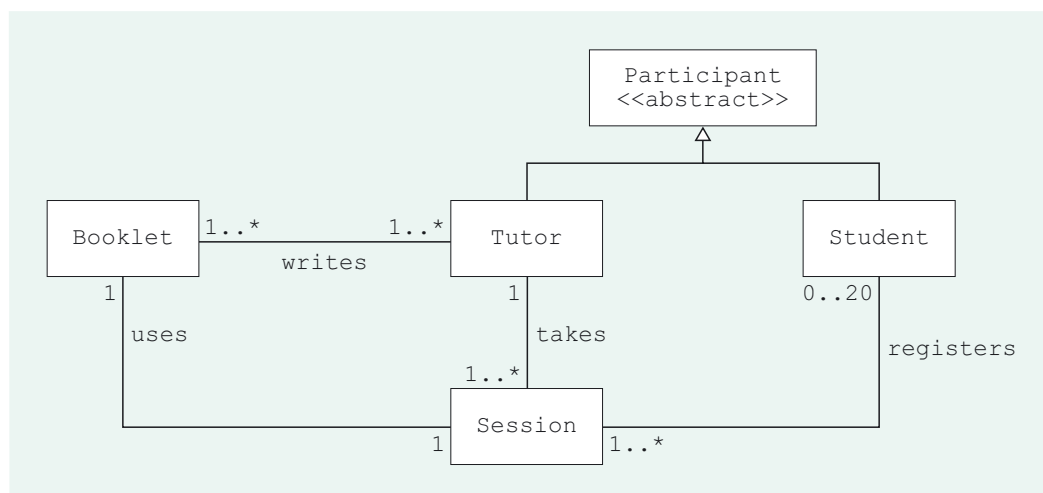


Figure 23 Class diagram for the Day School System

During the period of organisation for the day school it is possible that no student has (yet) registered for a particular session; hence the multiplicity of 0..20 at the `Student` end of `registers`.

A session is an example of an event modelled by a class.

In each of Exercises 10 and 11 we chose to model an event as a class – `Mating` in the Goat Breeding System and `Session` in the Day School System. But neither of these events has properties that need to be modelled as attributes.

Exercise 12

Study the class diagrams for the Goat Breeding and Day School Systems. Can you identify any similarities in the ways in which the `Mating` and `Session` classes fit into their respective class diagrams?

Discussion.....

`Mating` and `Session` each have associations with *three* distinct classes. `Mating` is associated with `BillyGoat`, `NannyGoat` and `Kid`; `Session` is associated with `Tutor`, `Student` and `Booklet`.

The two systems you have just explored illustrate a common situation – where an event involves three or more types of entity from the system domain. A single mating event in the Goat Breeding System involves a nanny-goat, a billy-goat and the offspring which result from that mating. An association between any two of the participant classes fails to capture this three-way relationship. In M256 an event that involves objects from more than two classes will always be modelled by a class, even if there are no properties associated with the event. Contrast this with the modelling of another event in the Day School System, a student registration. This involves only two classes – `Student` and `Session`. Furthermore, it has no properties of its own that need to be modelled as attributes. Registration is therefore modelled as a `registers` association between `Student` and `Session`.

What is really needed here is an association between *three* classes, but as associations can involve only two classes, an event class is introduced to link all three classes.

5.2 Summary of criteria

We will end this section by summarising the criteria to be considered when deciding whether an event is best represented by a class or an association.

- ▶ Does the event have any properties whose values need to be recorded? For example, in the Hospital System, had there been a need to record the date of a patient's treatment or the medication that a treatment involved, we would have modelled treatment by a `Treatment` class associated with `Doctor` and `Patient`.
- ▶ Does the event involve more than two other significant entities? For example, were nurses part of our system domain, a treatment might sometimes involve a nurse as well as the patient and the doctor. In that case too, a class would be necessary, since three objects would need to be linked rather than just two.
- ▶ Do different occurrences of the event involving the same entities need to be recorded? For example, if there were a need to record each treatment of a given patient by a particular doctor, a `Treatment` class would be appropriate, because an association only allows for one link between the same two objects.
- ▶ Where none of these situations pertains – when an event involves instances of only two classes, has no significant properties of its own, and where it is not necessary to record multiple occurrences involving the same two entities – the event should normally be modelled as an association between the two classes.

Exercise 13

Consider the following excerpt from a requirements document for an Election Campaign System. Identify all the events in the description that should be represented in the conceptual model, decide which of them should be modelled by a class and which by an association, and give reasons for your decisions. Sketch a class diagram, excluding multiplicities.

A system is required to model the organisation of an election campaign by a constituency political party. During the campaign, local activists carry out telephone canvassing and visit local community organisations from which the party hopes to gain support. The party records all the telephone calls and visits that have taken place. In the case of telephone calls the date of the call is recorded, together with a note of any issues mentioned by the constituent as likely to influence their voting behaviour. The party also organises a number of public meetings in different parts of the constituency. Each of these meetings is chaired by a member of the local committee, and involves at least two speakers, a local activist and a guest speaker selected from a panel drawn up by the national headquarters of the party.

You may assume that the following classes have already been identified as necessary: `LocalActivist`, `CommunityGroup`, `Constituent`, `CommitteeMember` and `GuestSpeaker`.

Discussion.....

The description mentions three relevant types of event – visits to local organisations, public meetings and telephone conversations.

- ▶ The visits involve only two classes – `LocalActivist` and `CommunityGroup`. There is no indication that specific information needs to be recorded about visits or that their number is significant, so it is appropriate to model them by a `hasVisited` association between `LocalActivist` and `CommunityGroup`.
- ▶ Public meetings, on the other hand, involve instances of three other classes: `LocalActivist`, `CommitteeMember` and `GuestSpeaker`. It is therefore appropriate to model this event as a `PublicMeeting` class.
- ▶ Telephone conversations between party activists and constituents involve only two classes, but we are told that certain details of the conversation – the date on which it took place and the issues raised by the constituent – are recorded. To model this, a `PhoneConversation` class is needed, which will have two attributes, date and issues.

A class diagram for the system (excluding multiplicities) is given in Figure 24.

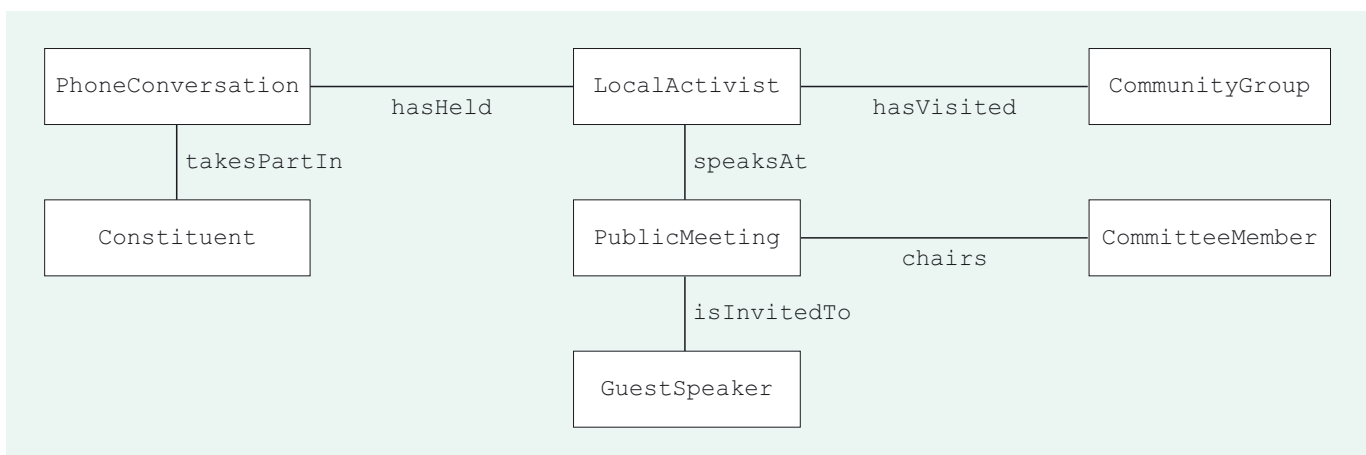


Figure 24 Class diagram for the Election Campaign System (excluding multiplicities)

6 Invariants

The aim in constructing a conceptual model is to capture all the relevant information about the system domain provided in the requirements document so that this model can form the basis for the structure of the system. To complete the process, there is one more component to add: invariants. In this section, you will learn:

- ▶ what is meant by an invariant;
- ▶ how to identify and express invariants.

6.1 What is an invariant?

An **invariant** is any condition on the objects of a conceptual model, their attribute values and links, which must hold for the requirements to be satisfied and the consistency of the model to be guaranteed. Such a condition represents a constraint on the real-world entities modelled (i.e. a constraint on the system domain).

We start by illustrating this concept with some examples from the Hospital System. You will find it easier to follow this discussion if you have the requirements document and the class diagram for the system in front of you. (These can be found in Sections 3 and 4 of the *Handbook*.)

Consider the following extracts from the requirements document for the system:

Each patient is ... under the care of a single team of doctors ...

A patient may be treated by any number of doctors but they must all be in the team that cares for the patient.

These extracts both imply invariants that must be true of our model at all times:

Each `Patient` object must be linked to one `Team` object – not zero, not more than one.

All the `Doctor` objects linked to a `Patient` object must be linked to the `Team` object to which that `Patient` object is linked.

We can express these two invariants more precisely as follows:

- 1 Each `Patient` object is linked to exactly one `Team` object.
- 2 If `aPatient` is any `Patient` object then any `Doctor` object linked to `aPatient` is also linked to the `Team` object which is linked to `aPatient`.

The first of these invariants imposes a condition on the *number of links* that are allowable between objects of two classes. Fortunately, some invariants, such as those concerned with the numbers of links allowed between an object of one class and object(s) of an associated class, may be evident from the class diagram, and so they would not normally need to be expressed in words. For example, this first invariant is represented by the multiplicity of 1 at the `Team` end of the `caresFor` association. However, other types of invariant cannot be shown using the M256 notation for class diagrams, and need to be included in the text which accompanies the diagram.

The second invariant above is one such example. It expresses a *dependency between links* of more than one association.

We will shortly discuss invariants which impose other types of condition on objects, attributes and links, but we will first explore the second invariant a bit more closely, by considering, in Exercise 14, how it imposes conditions on objects and links.

Exercise 14

Figure 25 shows two object diagrams, each modelling part of the hospital system domain at a particular time. One of the diagrams depicts a combination of objects and links which is invalid with respect to the second invariant above. Which diagram is invalid and why? (You may assume that all relevant links are shown.)

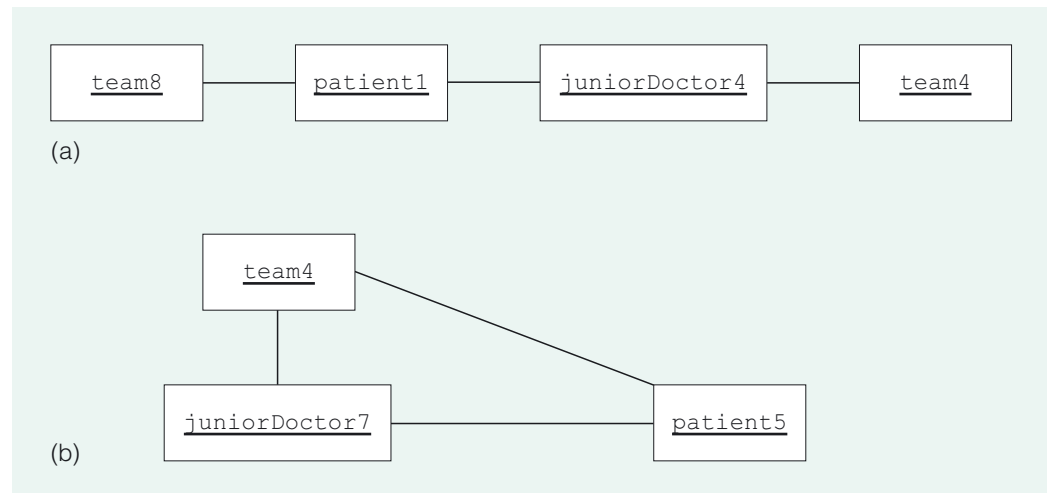


Figure 25 Object diagrams for the Hospital System, one of which depicts an invalid combination of objects and links

Discussion.....

Figure 25(a) is invalid. When a Patient object and a Doctor (in this case a JuniorDoctor) object are linked to each other, both must be linked to the same Team object, according to the second invariant, above.

Now consider the following extract from the requirements document for the Hospital System:

There are two types of ward, male wards and female wards. A ward can only have patients of the specified sex on it.

This constraint can be represented in the conceptual model as the following invariant:

The *sex* attribute of any Patient object has the same value as the *type* attribute of the linked Ward object.

This invariant expresses a condition on the relationship between attribute values of linked objects.

Exercise 15

Using the requirements document for the DVD Library System given in Subsection 3.2, and its completed class diagram in Figure 16, identify two invariants on the model of the DVD Library System. One invariant should relate to the number of allowable links between two objects, and the other to the attribute values of objects of a particular class. Try to express your invariants in terms of objects, attribute values and links, but do not spend too much time getting the wording precise.

Discussion.....

- 1 Each `Member` object is linked to no more than six `Loan` objects (there are other examples of this kind of invariant).
- 2 For each `Loan` object, the difference between the values of its `issueDate` and `returnDate` attributes is 3 days.

The examples above demonstrate that invariants can involve a number of different aspects of the conceptual model. Invariants involving additional types of constraint will become evident as we carry out a detailed analysis of the system domain for the Hospital System in Subsection 6.2, and consider the conditions needed to ensure the consistency of that model. First, though, we consider the forms in which invariants are included in the conceptual model.

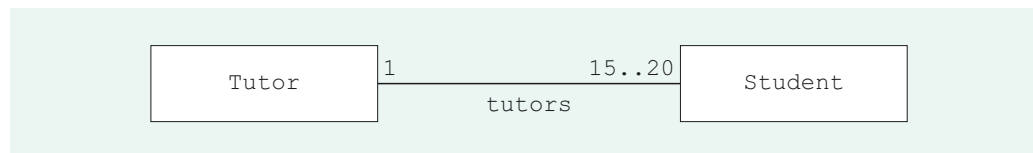
As the model will contribute to the design of the system, and ultimately its implementation, it is important that invariants are expressed clearly and without ambiguity. An invariant should be formulated as a statement that can be tested for its truth. As evidenced by some of the examples already considered, this is not always easy. Furthermore, there is often more than one way of expressing a particular invariant. For example, we could equally well have stated the second invariant in Exercise 15 as:

The `returnDate` attribute of each `Loan` object has a value that is three days later than that of its `issueDate` attribute.

Exercise 16

Figure 26 is a fragment from the class diagram for a university system. The `Tutor` class has attributes `name`, `staffNumber` and `region`, while the `Student` class has attributes `name`, `personalIdentifier` and `region`.

Write down an invariant which states that the tutor allocated to a student must be from the



same region as the student.

Figure 26 Fragment of class diagram for a university system

Discussion.....

The `region` attribute of each `Student` object has the same value as the `region` attribute of the linked `Tutor` object.

Or:

The `region` attribute of each `Tutor` object has the same value as the `region` attribute of each `Student` object to which that `Tutor` object is linked.

Exercise 17

Study the requirements document for the DVD Library System and see whether you can identify any further constraints on the system domain that require invariants involving just the values of attributes.

Discussion.....

The description sets out several constraints which will give rise to invariants concerned with the uniqueness of attributes:

- 1 No two members may have the same membership number.
- 2 No two DVDs may have the same number.
- 3 No two films may have the same title.

We could express the invariants noted in Exercise 17 in terms of objects and attribute values. If this were done for the first one it would read as follows:

If aMember1 and aMember2 are any two distinct Member objects, then the value of the number attribute of aMember1 is not the same as the value of the number attribute of aMember2.

Notice that the introduction of identifiers to refer to objects, e.g. aMember1 and aMember2, makes it easier to express the invariant clearly.

Invariants concerned with the uniqueness of attributes are so common that we will not generally ask you to identify them. However, this does not mean that they are not important. For simplicity, our models will include this information in the comments that accompany attributes in the class descriptions, as we have done for the name (in Ward) and code (in Team) attributes in the Hospital System.

Exercise 18

Attributes for the DVD objects and the Loan objects are not depicted because they are irrelevant in this exercise.

There is one further invariant for the DVD Library System. Study the two object diagrams in Figure 27, which model the situation regarding two particular films at a certain point in time. One of these illustrates a combination of objects and links which is invalid with respect to the numberAvailable attribute of Film, and the other a valid combination.

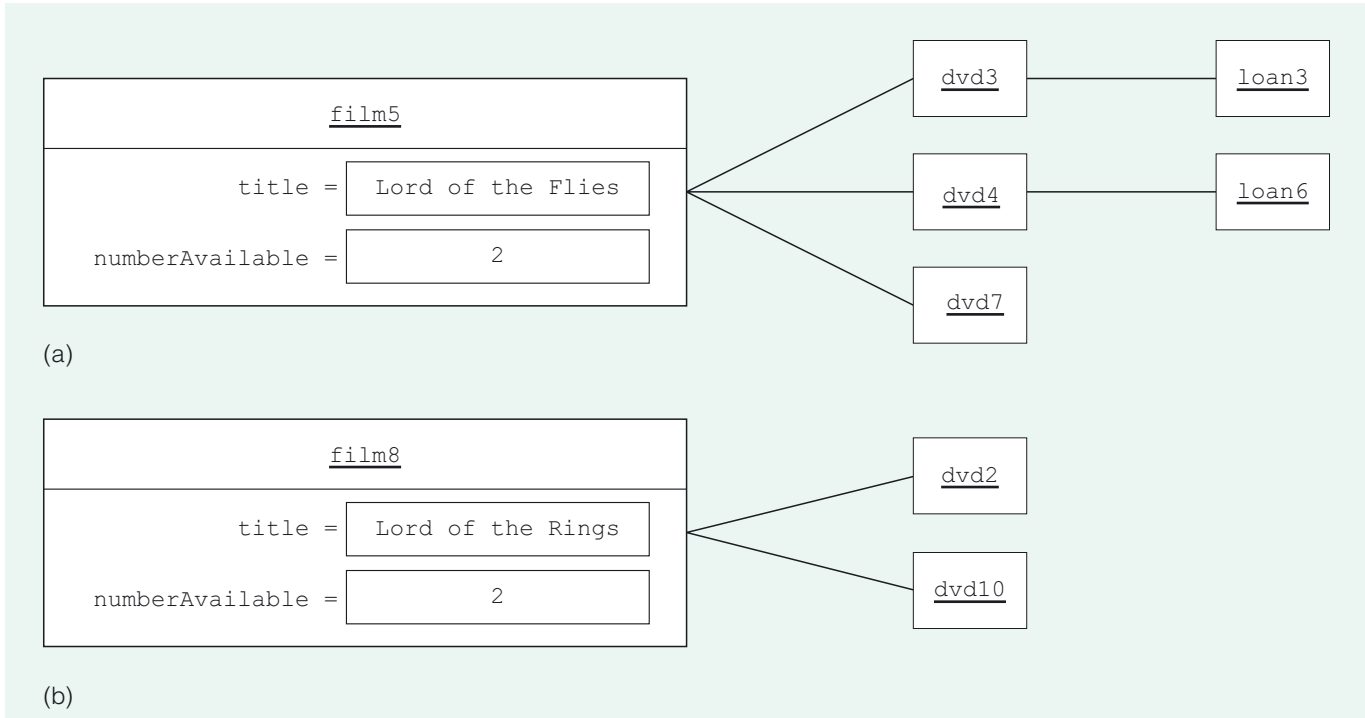


Figure 27 Object diagrams for the DVD Library System, one of which illustrates an invalid combination of objects and links

- (a) Identify which is the invalid combination and which is the valid combination, explaining your answer. (You may assume that all relevant links are shown.)
- (b) Write down a real-world constraint that requires an invariant to ensure that invalid combinations such as the one you have identified in part (a) cannot occur.

Discussion.....

- (a) Combination (a) is invalid. The diagram shows that the library has three copies of the film *Lord of the Flies*, of which two are on loan. The number available for loan is therefore 1, but the `numberAvailable` attribute of the `Film` object represented by `film5` has the value 2.
Combination (b) is fine, as the value of the `numberAvailable` attribute is consistent with the number of `DVD` objects which are linked to the `Film` object but not linked to any `Loan` object.
- (b) The real-world constraint is that the number of copies of a film that are available is the number of `DVDs` of that film which are not currently on loan.

The invariant noted in Exercise 18 can be written as:

For any `Film` object, the value of its `numberAvailable` attribute is equal to the number of `DVD` objects it is linked to which are not linked to a `Loan` object.

We will now complete our list of invariants for the Hospital System.

6.2 Remaining invariants for the Hospital System

Uniqueness of attribute values

There are two of these corresponding to the constraints that the team code of each team is unique and that the ward name of each ward is unique.

Constraints on the values of particular attributes

The requirements document states:

The system displays ... for each such junior doctor, their grade (1, 2 or 3).

This constraint could be included in the model as the following invariant:

The `grade` attribute of each `JuniorDoctor` object has a value which is either 1, 2 or 3.

Similarly, there are constraints on the sex of patients and the type of ward (which must be male or female in both cases). We take the view that invariants involving the uniqueness of attribute values or the allowable values of a single attribute are most simply and clearly expressed in a comment alongside that attribute in the class description, as we have done in the case of the descriptions for the Hospital System.

In addition to constraints arising from the requirements document, there can be restrictions on attribute values that are just 'common sense' – such as the date of birth of a patient not being later than the current date, or the number of free beds on a ward being non-negative. It is not necessary at this stage to identify such constraints explicitly.

Numbers of links between objects.

There are a number of invariants shown by the multiplicities of the associations in the class diagram. These do not need to be listed. However not all conditions on the number

of links between objects are evident from the class diagram. As we have discussed, the relationship between teams and the doctors that comprise them has been modelled as a single association between `Team` and `Doctor` rather than as two separate associations, one between `Team` and `JuniorDoctor`, and one between `Team` and `ConsultantDoctor`. Although the multiplicity at the `Doctor` end of `contains` correctly reflects the number of doctors that can be contained in a team, it does not fully reflect the allowable number of junior doctors and consultant doctors in a team.

Exercise 19

Using the first part of the answer to Exercise 15 as a model, write down two invariants, one which imposes a condition on the number of `ConsultantDoctor` objects that may be linked via `contains` to a `Team` object, and one which imposes a condition on the number of `JuniorDoctor` objects that may be linked to a `Team` object.

Discussion.....

- 1 Each `Team` object is linked via `contains` to exactly one `ConsultantDoctor` object.
 - 2 Each `Team` object is linked to at least one `JuniorDoctor` object.
-

In the first invariant given in the discussion to Exercise 19, notice the inclusion of 'via `contains`' to identify the relevant association connecting `Team` and `ConsultantDoctor`. This is because there are two associations between `Team` and `ConsultantDoctor` (namely, `isheadedBy` and `contains`), and it is therefore necessary to be explicit about the particular association being constrained. In other words, if in stating an invariant there is more than one association between two classes involved, it is necessary to specify an association name to clarify which association is being discussed.

The statement from the requirements document which gives rise to the two invariants in Exercise 19 is as follows:

Each team is headed by a consultant doctor who is the only consultant doctor in the team; the rest of the team are all junior doctors, at least one of whom must be at grade 1.

It indicates that for both the answers to Exercise 19 there is a need for an even tighter condition. The single consultant doctor in the team *must be the consultant doctor that heads the team*, and at least one of the junior doctors in the team *must be at grade 1*. The second invariant can be extended to reflect this, as follows:

Each `Team` object is linked to at least one `JuniorDoctor` object for which the `grade` attribute has a value of 1.

We will return to the first invariant in Exercise 21.

Relationships between attribute values

We have formulated an invariant reflecting the fact that the sex of a patient must be the same as the type of ward to which they are admitted (see the discussion following Exercise 14). We also know that each ward has a maximum capacity. The number of free beds must be consistent with this, leading to the following invariant:

For each `Ward` object, the value of its `numberOfFreeBeds` attribute is less than or equal to the value of its `capacity` attribute.

Exercise 20

The invariant we have just discussed constrains the relative values of two attributes of the same object. Write down a real-world constraint that requires a similar invariant involving attributes of another class in the Hospital System.

Discussion.....

A patient's age is the difference (in years) between the current date and their date of birth.

The invariant noted in Exercise 20 could be formulated as follows:

For each `Patient` object, the value of its `age` attribute is equal to the difference (in years) between the current date and the value of its `dateOfBirth` attribute.

Relationships between attribute values and links

Before Exercise 20, we discussed the relationship that needs to be preserved between the capacity of a ward and the number of free beds on that ward. A related factor is the number of patients already on the ward. The invariant should be extended to take account of this, as follows:

For each `Ward` object, the value of its `numberOfFreeBeds` attribute is equal to the value of its `capacity` attribute minus the number of `Patient` objects to which it is linked.

Relationships between links

You have already seen an invariant involving links of the associations `caresFor`, `hasTreated` and `contains`, connecting the classes `Team`, `Patient` and `Doctor`.

See the second invariant at the beginning of Subsection 6.1.

A similar invariant arises in connection with the associations `caresFor`, `isResponsibleFor` and `isHeadedBy`. The requirements state:

Each patient ... is under the care of a single team of doctors; the consultant who heads that team is responsible for the patient.

The relevant fragment of the class diagram is shown in Figure 28.

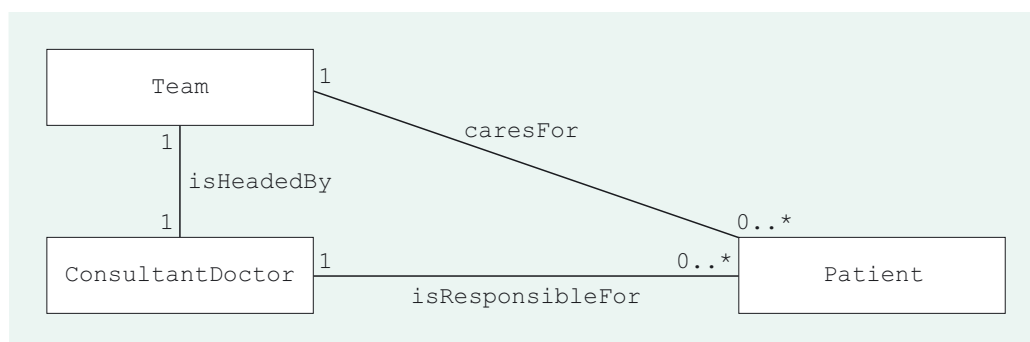


Figure 28 A loop giving rise to an invariant

The diagram represents the facts that a patient is cared for by a single team and is the responsibility of a single consultant, but there is nothing in the diagram to indicate that the consultant who is responsible for the patient must head the team that cares for that patient. This is clearly important, and we need to include it in the text that forms part of our model.

Remember that the association names are needed here to avoid ambiguity.

We can state the invariant as follows:

The `ConsultantDoctor` object linked to any `Patient` object via `isResponsibleFor` is linked via `isHeadedBy` to the `Team` object to which that `Patient` object is linked.

You can see from Figure 28 that the three associations involved in this invariant form a loop, as was the case with the invariant involving the `caresFor`, `hasTreated` and `contains` associations at the beginning of Subsection 6.1. Invariants are often (but not always) required when a number of associations in a class diagram form a loop.

Exercise 21

In the discussion following Exercise 19 we concluded that an invariant was required to reflect the constraint that the consultant doctor in a team must be the consultant that heads the team.

- Which associations are involved in this invariant?
- Write down the invariant required.

Discussion

- The associations involved are `contains` and `isHeadedBy`.
- The invariant can be written as follows:

The `ConsultantDoctor` object linked to a `Team` object via `isHeadedBy` is linked to the same `Team` object via `contains`.

The invariant which was the subject of Exercise 21 also involves a loop as shown in Figure 29. However, although there are three classes shown in this diagram (`Doctor`, `Team` and `ConsultantDoctor`), the loop is between only two classes (`Team` and `ConsultantDoctor`). This is because `ConsultantDoctor` inherits the `contains` association from `Doctor` (since it is a specialisation of `Doctor`), and consequently is in a loop with `Team` via the `isHeadedBy` and `contains` associations.

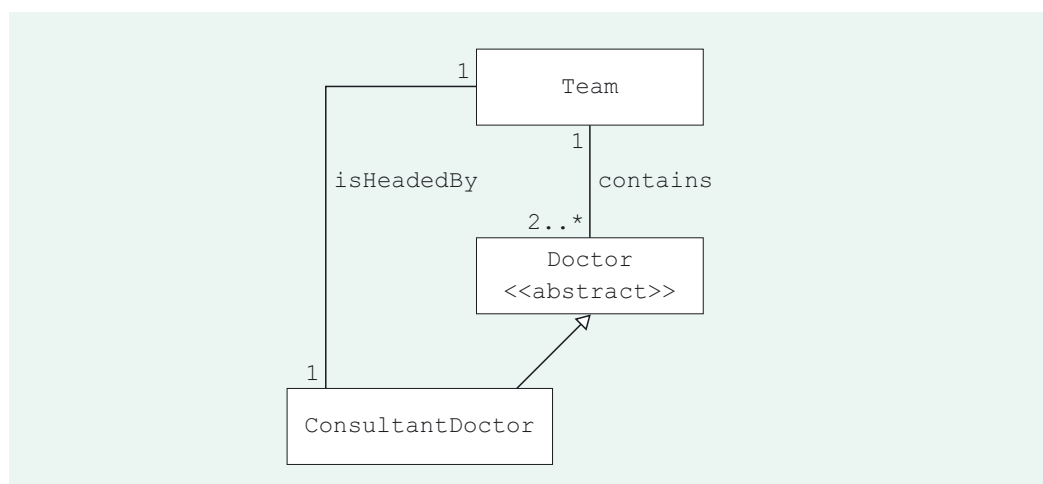


Figure 29 The `Team` and `ConsultantDoctor` classes and the associations between them

Exercise 22

Look again at the description of the Goat Breeding System given in Exercise 10 and the class diagram given in the discussion of the exercise.

(a) Identify an extract from the description which gives rise to one or more invariants that cannot be deduced from the class diagram.

(b) State the invariant(s).

Discussion.....

(a) The invariant(s) can be identified from the following extracts from the description.

A billy-goat mates several times during the current season but only with nanny-goats that are in the same herd as the billy-goat. A mating can produce several offspring, which – of course – are kids and are in the same herd as their parents.

In other words if a billy-goat and a nanny-goat mate, they and their offspring must be in the same herd.

(b) Here is a single invariant covering the relationships between all three kinds of goat:

All the Goat objects linked to each Mating object are linked to the same Herd object.

There are a number of other ways of expressing this invariant, for example:

If aMating is any Mating object, then the NannyGoat and BillyGoat objects linked to aMating, and any Kid objects linked to aMating, are linked to the same Herd object.

Alternatively the invariant could be formulated as two separate invariants, each involving two of the associations isFemalePartnerIn, isMalePartnerIn and produces, together with the association contains.

Because Goat generalises NannyGoat, BillyGoat and Kid, all their instances are also instances of Goat and so are linked to instances of Herd via contains.

Some of the invariants we have identified above could have been avoided by choosing an alternative way of modelling the system domains concerned. For example, representing the fact that each team of doctors includes both a consultant doctor and a junior doctor could have been achieved by replacing the association between Team and Doctor with two separate associations, one between Team and ConsultantDoctor, and one between Team and JuniorDoctor. We could even have introduced a separate class for grade 1 junior doctors and connected it to Team with a third association. However, this alternative model would have introduced more complexity into our class diagram, and would still have required some invariants, so would not have provided any advantage.

It is an understanding of what is significant in the system domain that should inform decisions as to which classes and associations are most appropriate. However, as we have already said, the construction of a conceptual model is not an exact science. The best advice at this stage of the modelling process is to choose the simplest option that accurately reflects the situation in the real-world system domain; any appropriate changes or improvements to the ensuing model of the system structure can be considered later in the development.

Having specified invariants for the Hospital System, we include these in the text which, together with the class diagram, constitutes our model. A consolidated set of the invariants for the Hospital System is given in Section 4 of the *Handbook* and in the final section of this unit.

The invariants for the DVD Library System can be found in Section 9 of the *Handbook*.

Exercise 23

Consider the Day School System described in Exercise 11, and the class diagram given in the solution (Figure 23).

- (a) Identify an excerpt from the requirements document which gives rise to an invariant.
- (b) State this invariant.

Discussion.....

- (a) 'Each booklet is written by a number of tutors ... one of whom must be the tutor who is taking the session that uses the booklet.'
- (b) The Tutor object linked to any Session object is linked to the Booklet object linked to that Session object.

As usual, there are other, equally correct, ways of expressing this invariant.

Exercise 24

Consider the Election Campaign System described in Exercise 13. Suppose that the conceptual model for the system contained the following invariant:

Each LocalActivist object linked to a PublicMeeting object is also linked to two or more CommunityGroup objects.

Write down the real-world constraint that this invariant would represent.

Discussion.....

A local activist who speaks at a public meeting must have visited at least two community groups.

6.3 Use of the term 'constraint'

In Unit 2 the term 'constraint' is used to mean a limitation on the development of the system, or on the system itself that is not wholly a behavioural requirement being, for example, related to the hardware and/or the language that is to be used to implement the system. For example, a statement of the time an action has to take or the amount of memory that is available for the system is a constraint in this sense.

In Units 3 and 4 onwards the term 'constraint' is used slightly differently, to mean a real-world condition that imposes a restriction on the actual behaviour of the system that is independent of the hardware or language used to implement it. Thus a constraint in this sense is a behavioural requirement and might be modelled by an invariant or by other aspects of a conceptual or structural model.

It should always be apparent from the context what meaning is attached to the term.

In this section, you have learnt what is meant by an invariant, and how to identify and express them. In the process, a list of the invariants for the Hospital System has been drawn up. In the next, short section you will explore some of the implications of invariants for other aspects of the model.

7

Derived attributes and associations

In this section, you will learn:

- ▶ that some attributes and associations in a conceptual model are derivable from others;
- ▶ how invariants are involved in this process.

7.1 Using invariants to derive attributes

The need for invariants in a conceptual model arises from the fact that the entities in the system domain may be interdependent in a way that is not evident from their representation in the conceptual model's class diagram and class descriptions. Where there are constraints on some aspect of the system domain, an invariant spells out the conditions that ensure that this constraint is reflected in the model.

Some invariants express a relationship between two or more elements of the model. For such invariants, it may be possible, given the values of one or more elements, to work out the value of some other element. For example, given the invariant

For each `Patient` object, the value of its `age` attribute is equal to the difference (in years) between the current date and the value of its `dateOfBirth` attribute.

and knowing both the current date and the value of a `Patient` object's `dateOfBirth` attribute, it is possible to work out the value of the `Patient` object's `age` attribute.

You might justifiably ask why the `Patient` class needs both the attributes `age` and `dateOfBirth`. The simple answer is that they are both explicitly mentioned in the requirements document.

However, in our final system it may not be necessary to implement both these attributes as instance variables. This is not a decision that can be made until the analysis process has advanced beyond the conceptual model. It will depend on the results of a detailed analysis of the behaviour of the system, which is yet to be carried out. This analysis may reveal that one or other of the attributes does not need to be implemented explicitly, or it may be that the context in which each is required indicates the need to retain both. The decision may also depend on other implementation issues, such as whether the implementation language and hardware contain facilities for accessing the current date in a suitable form.

At this stage of the development process the model needs to reflect *all* the aspects of the system domain implied by the requirements document. This document refers to both the age and the date of birth of patients, so both these properties need to be modelled as attributes of the `Patient` class. However, it is also necessary to look out for situations where one aspect of the model can be deduced from another and to flag this up as part of our model. An attribute whose value can be deduced from other element(s) of the model is known as a **derived attribute**. Derived attributes are indicated in the UML by preceding their names with a slash, /. So we modify the entry for the `Patient` class in the class descriptions given at the end of Subsection 3.3 as follows:

Class	Patient	A patient in the hospital
Attributes		
	name	The name of the patient
	sex	The sex (M or F) of the patient
	dateOfBirth	The date of birth of the patient
	/age	The age of the patient in years

Exercise 25

Identify a derived attribute for the DVD Library System, and provide a class description for the class to which this attribute belongs.

Discussion.....

In the `Loan` class, `returnDate` can be derived from `issueDate` on the basis of the invariant discussed in Exercise 15, which stated that the difference between these two attributes must be three days.

The class description is as follows:

Class	Loan	The loan of a DVD
Attributes		
	issueDate	The date of issue of the loan
	/returnDate	The date by which the loaned DVD must be returned

Alternatively, `issueDate` could be derived from `returnDate`. (But note that it would not be possible to derive a patient's date of birth from their age in years.)

Another attribute value that can be deduced from other aspects of the information in the Hospital System is the number of free beds on a ward. In the discussion following Exercise 20, we formulated the following invariant, which expresses how the number of free beds on a ward can be derived from other aspects of the model:

For each `Ward` object, the value of its `numberOfFreeBeds` attribute is equal to the value of its `capacity` attribute minus the number of `Patient` objects to which it is linked.

From this invariant we see that the value of a `Ward` object's `numberOfFreeBeds` attribute can be calculated from the value of that `Ward` object's `capacity` attribute and the number of links the `Ward` object has to `Patient` objects. So the `numberOfFreeBeds` attribute should be marked as derived.

The `sex` attribute of `Patient` should also be marked as derived, since it can be obtained from the `type` (male or female) of the `Ward` object the `Patient` object is linked to. This derivation may seem counter-intuitive – it does not seem sensible to work out the sex of a patient from the type of the ward they are on. Nonetheless it is conceptually possible, so it should be noted.

Exercise 26

Is the `numberAvailable` attribute of the `Film` class in the DVD Library System derived? If so, how?

Discussion.....

Yes, the `numberAvailable` attribute is derived.

It can be calculated from the following invariant:

For any `Film` object, the value of its `numberAvailable` attribute is equal to the number of `DVD` objects it is linked to which are not linked to a `Loan` object.

Exercise 27

Consider the following invariant for the DVD Library System (which is actually captured in the comment on the `number` attribute of the `Member` class):

If `aMember1` and `aMember2` are any two distinct `Member` objects, then the value of the `number` attribute of `aMember1` is not the same as the value of the `number` attribute of `aMember2`.

Does this invariant indicate that any attribute of the `Member` class can be derived?

Discussion.....

No. Given the value of the `number` attribute for one `Member` object, this invariant specifies what the value of any other object may *not* be, not what it is. Invariants involving attributes are not always an indication of derived attributes. It depends on the nature of the condition specified.

You have seen that where an invariant involves the values of attributes, the value of one of those attributes *may* be derivable from values of other attributes(s) and/or link(s) involved in the invariant. However, it is not necessarily the case that it is the invariant that leads you to identify the derived attribute. It could be that you recognise that an attribute can be derived before you consider invariants, or that you identify both aspects of the model in parallel. The important point is to be aware of the relationship between them: where you have a derived attribute, you will also have an invariant. However, the reverse is not necessarily the case. As you saw in Exercise 27 above, an invariant involving attributes does not necessarily indicate that it is possible to derive one of those attributes.

7.2 Using invariants to derive associations

Just as there is a relationship between invariants involving attributes and derived attributes, so it is the case that invariants involving links of different associations *may* indicate that one of those associations can be derived from the others involved in the invariant.

To explore the sorts of interrelationship that can occur between associations, we will start by revisiting the invariant which constrains the allowable links between the classes `Patient`, `Team` and `ConsultantDoctor`.

Here is the invariant which was formulated in the previous section:

The `ConsultantDoctor` object linked to any `Patient` object via `isResponsibleFor` is linked via `isHeadedBy` to the `Team` object to which that `Patient` object is linked.

This invariant involves the associations `caresFor`, `isResponsibleFor` and `isHeadedBy`, as shown in Figure 30.

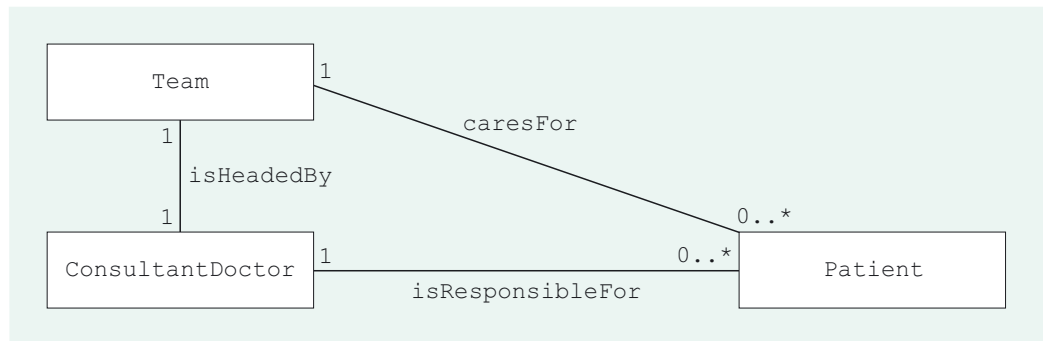


Figure 30 Extract from the Hospital System class diagram

Now consider the case of a particular `Patient` object, say `patient6`, linked via the association `caresFor` to a `Team` object, say `team3`, which represents the team that cares for the patient. `team3` will in turn be linked via `isHeadedBy` to a `ConsultantDoctor` object, say `consultantDoctor5`, which represents the consultant doctor who heads that team. What the invariant expresses is that the particular `ConsultantDoctor` object to which `patient6` must be linked via `isResponsibleFor` must be `consultantDoctor5`, as shown in the object diagram in Figure 31.

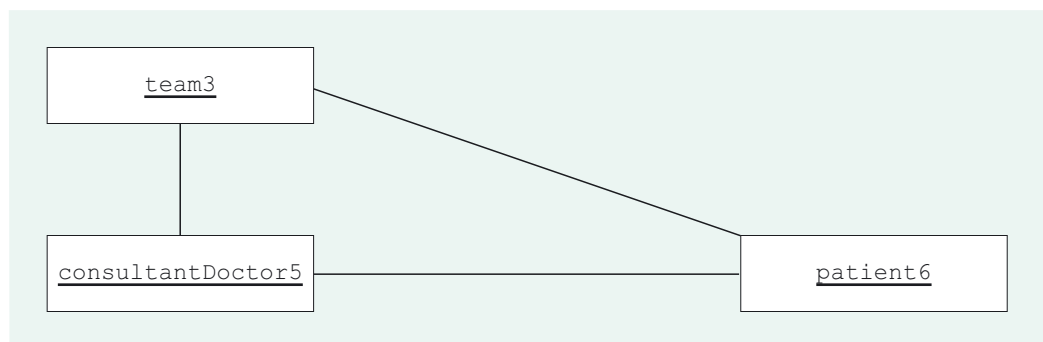


Figure 31 Links between `Patient`, `Team` and `ConsultantDoctor` objects

A situation similar to that depicted in Figure 31 holds for *all* `Patient` objects. Given any `Patient` object representing a particular patient, it is always possible to identify the `Team` object representing the team which cares for the patient by traversing the `caresFor` link, and then the `ConsultantDoctor` object representing the consultant doctor who heads that team by traversing the `isHeadedBy` link. As this must be the consultant doctor who is responsible for the patient, it is therefore possible to infer, for any `Patient` object, the particular `ConsultantDoctor` object representing the consultant doctor who is responsible for the patient, i.e. the link between the `Patient` object and the `ConsultantDoctor` object. In other words, it is possible to derive `isResponsibleFor` from `caresFor` and `isHeadedBy`.

The association `isResponsibleFor` is called a **derived association**. As with a derived attribute, the name of a derived association in a class diagram is preceded by a slash, /, as illustrated in Figure 32.

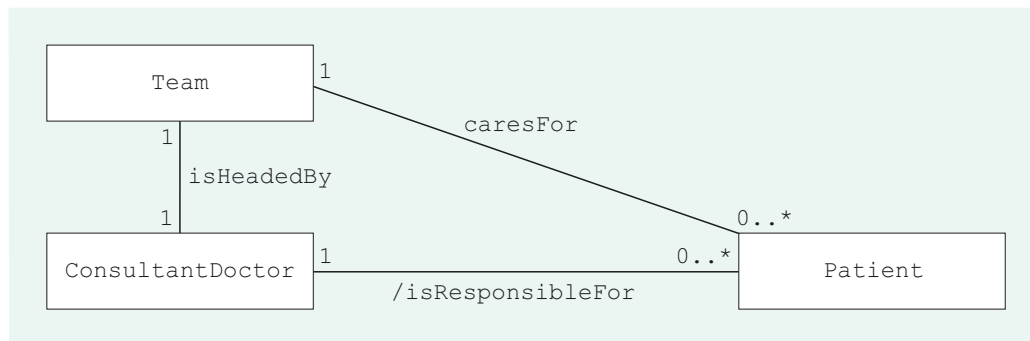


Figure 32 A derived association

As with derived attributes, you might wonder why the derived association `isResponsibleFor` is not simply removed; it is, in a sense, redundant. There are two reasons. The first is that `isResponsibleFor` models a relationship that is explicitly mentioned in the requirements document – the responsibility of consultant doctors for patients. It is important, at least at this stage of the development process, that this relationship is explicitly modelled.

The second reason for retaining `isResponsibleFor` is that its designation as a derived association is somewhat arbitrary. Consider the following invariant involving `isResponsibleFor`, `caresFor` and `isHeadedBy`, which is equivalent to the one above:

The `Team` object linked to a given `Patient` object is linked via `isHeadedBy` to the `ConsultantDoctor` object to which that `Patient` object is linked via `isResponsibleFor`.

Working from this version of the invariant we could equally well have said that the `caresFor` association could be derived from the `isResponsibleFor` and `isHeadedBy` associations, and that it was therefore `caresFor` that was redundant rather than `isResponsibleFor`. It is only when the detailed behaviours of the system are analysed in subsequent stages of the development process that we can decide which, if any, of the associations related in this way is actually redundant. However, it is important that the class diagram in the conceptual model should indicate any derived associations that we have identified.

For an association to be derivable, *all* the links of the association must be derivable. With this in mind, can the association `isHeadedBy` be derived from the associations `caresFor` and `isResponsibleFor`?

We will answer this question in terms of the multiplicities of the associations involved. (It will help to refer to Figures 31 and 32.)

Suppose a `Team` object (such as `team3` in Figure 31) is linked to a `Patient` object (`patient6`). Then the `ConsultantDoctor` object (`consultantDoctor5`) linked to `patient6` via `isResponsibleFor` must also be linked, via `isHeadedBy`, to `team3`. In other words, *if* there is a `Patient` object linked to a `Team` object, then it is possible to derive the link between the `Team` object and the `ConsultantDoctor` object.

Consider, however, the situation depicted in Figure 33: a consultant doctor represented by `consultantDoctor7` heads a team represented by `team2`. Suppose also that this particular team does not yet have any patients under its care. This is entirely possible, as indicated by the multiplicity of `0..*` at the `Patient` end of `caresFor`. In this case, the consultant represented by `consultantDoctor7` would not be responsible for any patients.



Figure 33 A Team object which is not linked to any Patient objects

We could equally have traversed the links the other way and said, locate first a Patient object linked to consultantDoctor7 and then the Team object linked to that Patient object.

To derive the link between consultantDoctor7 and team2 shown in Figure 33, it would be necessary to locate first a Patient object linked to team2 via the caresFor association and then the ConsultantDoctor object linked to that Patient object via isResponsibleFor. Neither of these links exists. Therefore it is not possible to derive the link shown in Figure 33.

In summary, we have shown that the link between consultantDoctor5 and team3 in Figure 31 could be derived from the other links in the diagram, but that the link between consultantDoctor7 and team2 in Figure 33 could not (because there are no other links from which it can be derived).

We therefore conclude that because it is not possible to derive *all* the links of the isHeadedBy association, isHeadedBy cannot be derived from caresFor and isResponsibleFor.

Reviewing this example, it is instructive to note that what we did to show that isHeadedBy is not derivable was to consider a possible combination of objects and links, which conformed to our model, but where an isHeadedBy link could not be deduced from other information. This is a generally useful approach, as you will see in the next exercise.

Exercise 28

Study the invariant involving the three associations hasTreated, contains and caresFor given below and the associated extract from the class diagram shown in Figure 34. Decide whether the hasTreated association can be derived from the other two associations, giving an explanation for your answer.

If aPatient is any Patient object, then any Doctor object linked to aPatient is also linked to the Team object which is linked to aPatient.



Figure 34 The associations contains, caresFor and hasTreated

Discussion.....

The association cannot be derived. The hypothetical situation portrayed in Figure 35 will demonstrate why.

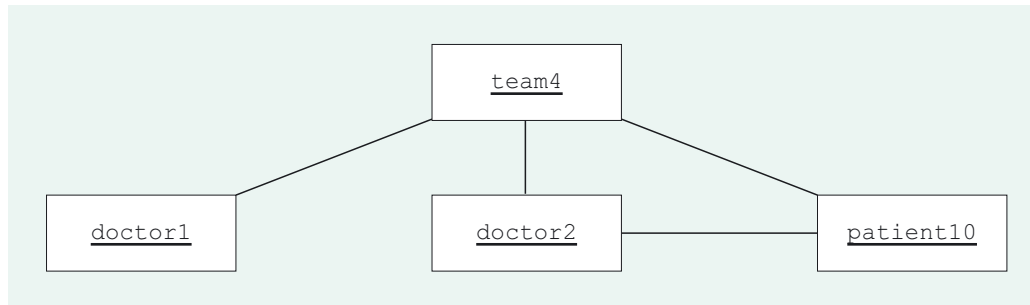


Figure 35 Links between Team, Patient and Doctor objects

Figure 35 depicts the situation where, at some point in time, the patient represented by `patient10` is cared for by the team represented by `team4`, and has been treated by one of the doctors in that team, represented by `doctor2`. This situation is perfectly acceptable within the conditions imposed on the model. The task is to decide whether, given the links from `team4` to `doctor2` and to `patient10`, it is possible to derive the link between `doctor2` and `patient10`.

Having traversed the `caresFor` link from `patient10` to `team4`, you could traverse the `contains` links to identify all the `Doctor` objects linked to `team4`, that is, all the `Doctor` objects that are eligible to treat `patient10`. However it is not possible to deduce that `doctor2` has treated `patient10` and that `doctor1` has not.

In summary, a `Doctor` object linked to a `Team` object that is linked to a `Patient` object is not necessarily linked to that `Patient` object.

Now consider whether it is possible to derive the `contains` association from `caresFor` and `hasTreated`. Given a `Team` object, if there are `Patient` objects linked to the `Team` object, that is, if there are patients cared for by the team represented by the `Team` object, then these `Patient` objects can be identified by traversing the relevant `caresFor` links, while any `Doctor` objects corresponding to doctors who treated these patients can be identified by traversing the relevant `hasTreated` links. These `Doctor` objects must be linked to `Team` objects via `contains`. However, `Doctor` objects corresponding to doctors in the team who have *not* treated any patients cannot be identified in this way. Consequently, although some of the `contains` links are deducible, not *all* are, and therefore the `contains` association is not derivable from `caresFor` and `hasTreated`.

Similarly, the `caresFor` association cannot be derived from `hasTreated` and `contains`.

What the above discussion has shown is that an invariant involving a loop indicates that it *may* be (rather than *will* be) possible to derive one of the associations which make up the loop.

Note that a loop may consist of more than three classes and associations.

In this section, you have learnt what is meant by derived attributes and associations, how invariants are used in the derivations, and how they are denoted in the conceptual model. This completes the task of conceptual modelling, and in the next section, all the parts will be drawn together.

8

Conceptual model for the Hospital System

We are now in a position to produce a conceptual model for the Hospital System.

In M256 a conceptual model comprises the following:

- 1 a class diagram showing:
 - ▶ the classes that have been identified as appropriate;
 - ▶ any generalisation relationships between the classes;
 - ▶ any abstract classes;
 - ▶ the associations between the classes, together with their multiplicities;
 - ▶ any derived associations;
- 2 associated text comprising:
 - ▶ for each class, a class description listing the attributes, together with the identification of any derived attributes, and brief comments indicating:
 - the purpose of the class and of each attribute;
 - if the class is abstract;
 - any generalisation/specialisation relationships;
 - any restrictions on attribute values where relevant, including uniqueness constraints;
 - ▶ any invariants not evident from the class diagram or class descriptions.

In producing the conceptual model we have aimed to create an appropriate representation of the system domain. However, we have had to make a number of decisions, for example, about which attributes and associations to label as derived. If we had made different decisions we would have produced a slightly different model, but one which might have been equally good at this stage. There is no one correct model (except perhaps in extremely trivial cases), and there are many debates, even among experts, about whether one model is better than another. Our concern is only that you should understand the concepts well enough to be able to produce a reasonable conceptual model. The examples you will study in the next unit, and that we will ask you to attempt in assignments, will be tightly specified and simple enough to ensure that you do not have too many possibilities to choose from.

All the aspects of the conceptual model for the Hospital System have now been considered. However, before presenting this conceptual model, recall the earlier discussion about the role of the conceptual model in forming a starting point for constructing the system. The conceptual model represents the system domain – the part of the real world relevant to the system. Since a key object-oriented principle is that software objects can usefully mirror real-world entities, the structure set out in the conceptual model is a sensible starting point for the structure of the system. Thus the *conceptual model*, given below, can be taken as the *initial structural model*: an initial representation of the software. That is, we will initially view the Hospital System as having classes such as *Patient* and *Doctor*, whose instances and the links between them:

- ▶ represent real-world entities (patients and doctors etc.), and the connections between them;
- ▶ are subject to invariants, just as their real-world equivalents are subject to real-world constraints.

As we move forward from this unit into the next phases of development our perspective will become that of the software rather than of the system domain, and we will consider the conceptual model constructed in this unit as our initial structural model, and refer to it as such. This initial structural model will be the first in a series of **structural models**, which represent, in increasing detail, the system in terms of classes and the relationships between them.

Here is the complete model, which is also set out in Section 4 of the *Handbook*, under the heading *Hospital System initial structural model*.

The initial structural model for the DVD Library System is set out in Section 9 of the *Handbook*.

Class diagram

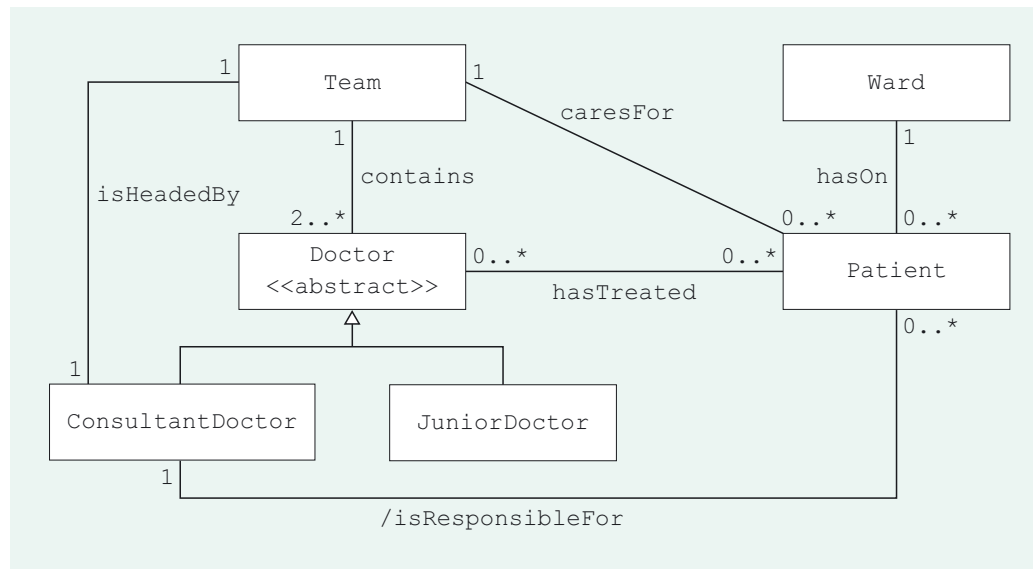


Figure 36 Class diagram for the Hospital System

Class descriptions

Class	Ward	A hospital ward
Attributes		
	name	The unique name of the ward
	type	Whether the ward is for male or female (M or F) patients
	capacity	The maximum number of patients that can be on the ward at any one time
	/numberOfFreeBeds	The number of free beds on the ward
Class	Patient	A patient in the hospital
Attributes		
	name	The name of the patient
	/sex	The sex (M or F) of the patient
	dateOfBirth	The date of birth of the patient
	/age	The age of the patient in years

Class	Team	A team of doctors
	Attributes	
	code	The unique code of the team
Class	Doctor	A doctor at the hospital Abstract: generalises JuniorDoctor and ConsultantDoctor
	Attributes	
	name	The name of the doctor
Class	JuniorDoctor	A junior doctor at the hospital Specialises Doctor
	Attributes	
	grade	The grade (1, 2 or 3) of the junior doctor
Class	ConsultantDoctor	A consultant doctor at the hospital Specialises Doctor
	Attributes	
	None	

Invariants

- 1 The sex attribute of any Patient object has the same value as the type attribute of the linked Ward object.
- 2 For each Patient object, the value of its age attribute is equal to the difference (in years) between the current date and the value of its dateOfBirth attribute.
- 3 For each Ward object, the value of its numberOfFreeBeds attribute is equal to the value of its capacity attribute minus the number of Patient objects to which it is linked.
- 4 Each Team object is linked to at least one JuniorDoctor object for which the grade attribute has a value of 1.
- 5 The ConsultantDoctor object linked to a Team object via isHeadedBy is linked to the same Team object via contains.
- 6 The ConsultantDoctor object linked to any Patient object via isResponsibleFor is linked via isHeadedBy to the Team object to which that Patient object is linked.
- 7 If aPatient is any Patient object, then any Doctor object linked to aPatient is also linked to the Team object which is linked to aPatient.

Optional exercise

Our analysis of the Hospital System has been conducted in a rather artificial manner. We attempted to identify all the classes before identifying their attributes and then the associations between them. Similarly, we did not consider invariants until we had formed the class diagram and class descriptions, and did not identify derived attributes and associations until we had found all the invariants. In practice, you would use your understanding of the system domain to carry out these activities in parallel as you worked through the requirements document.

Here is a worthwhile exercise which you can do now, if you have time, or as part of your revision later in the course. Go back to the requirements document for the Hospital System and work through it a sentence or two at a time, building up the class diagram (classes, associations and multiplicities) and class descriptions as you go, and noting any possible invariants and derived associations when they arise. You may be surprised at how easy it is to get most of the model correct in a short time. The areas requiring most thought are usually invariants and derived associations.

9

Summary

This unit has introduced you to the ideas and techniques involved in forming a conceptual model of a system domain and in particular a conceptual model for the Hospital System.

Starting with a requirements document for a system, you have worked through the processes necessary to produce the conceptual model. The unit has also introduced some of the UML notation for class and object diagrams.

In producing the conceptual model, you have learnt how to analyse a requirements document to identify classes and associations appropriate for modelling the system domain, together with their respective attributes and multiplicities. You also learnt about the value of generalisation relationships in capturing the characteristics common to related classes and how this might lead to the identification of abstract classes.

You have seen that objects, attribute values and links may be subject to constraints in the form of invariants, and that attributes and associations can sometimes be derived from other attributes and associations, using invariants.

Choices have to be made during this phase of the software development process, and you have seen that there is no one correct conceptual model for a system.

Finally, the unit ended by bringing together all the pieces of the Hospital System's conceptual model, which in the remaining phases of the development process (starting in *Unit 6*) will be treated as the initial structural model for the software system itself.

In *Unit 4* you will be applying the ideas and techniques learnt in this unit to the task of producing a conceptual model for the Shopping System, introduced in *Unit 2*.

LEARNING OUTCOMES

After studying this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ list the components that make up a conceptual model of a system domain, and describe briefly the form and purpose of each component;
- ▶ explain why a conceptual model forms the basis for modelling the structure of the system, i.e. gives an initial structural model of the system;
- ▶ use various techniques and guidelines to identify an appropriate set of classes and associations from a requirements document, including their attributes and multiplicities;
- ▶ identify situations where a generalisation relationship between classes exists, and decide on appropriate parent and child classes, and their attributes and associations;
- ▶ use various criteria to decide whether an event should be modelled as a class or as an association;
- ▶ identify invariants and express them in terms of classes, associations and attributes, and distinguish between valid and invalid combinations of objects and links;
- ▶ identify and reason about derived attributes and associations;
- ▶ draw simple object and class diagrams using the diagrammatic notations introduced in this unit.

Glossary

abstract class A class which cannot be instantiated, i.e. one which has no instances of its own but only instances of its child classes. An abstract class defines properties which are common to its child classes. It should not be confused with a class representing abstract entities.

abstract entity An entity that has an existence, but does not correspond to a tangible entity. For example, the film *Citizen Kane* is an abstract entity, while the DVD which records the film is a tangible entity.

association An association represents the links that may exist between the objects of two classes. These links have a common meaning, and represent a particular relationship (connection) between these objects that is significant for the system domain.

attribute An attribute is a property of an object. All objects of a class have the same attributes.

candidate class A class that may be needed to model a category of real-world entities from the system domain.

child class A class that is a specialisation of some other class, known as its parent class; a subclass.

class (in the conceptual model) A class models a category of real-world entities from the system domain.

class description A formal description of a class within a conceptual model or a structural model.

class diagram Class diagrams are used to show structural aspects of the system domain and of a system at various stages in its development. As a model of the *system domain*, a class diagram depicts conceptual classes and associations connecting them. As a model of the *system*, a class diagram depicts the software classes that are involved in the system and the associations connecting them.

conceptual model The model which represents the significant entities in the system domain and the connections between them as a collection of classes and their attributes, associations and invariants. The conceptual model of the system domain is taken to be the initial structural model for the software system to be developed. It consists of a class diagram, class descriptions and invariants.

concrete class A class that, unlike an abstract class, may have instances that are not instances of any of its child classes.

constraint (on the system domain) A limitation on the entities in the system domain, their properties and relationships.

derived association An association all of whose links can be derived using other elements of the model.

derived attribute An attribute all of whose values can be derived using other elements of the model.

event (in the system domain) Any significant circumstance, episode, interaction, happening or incident, e.g. deliveries, registrations, bookings, enrolments.

generalisation A relationship between classes that captures the similarities and differences between two related classes. Saying that class A is a generalisation of class B means that the common (similar) structure and behaviour is defined by class A, and the distinctive (different) structure and behaviour is defined by class B. It means that an instance of class B is substitutable for an instance of class A.

inherit Take on the properties (attributes and associations) of a parent class.

initial structural model The initial structural model is the conceptual model viewed as the first model of the structure of the proposed software system. In other words, the conceptual and initial structural models 'look' the same but have different perspectives: one models the system domain; the other is taken as the first model of the software system itself.

instance Refers to a particular object of a class or a particular link of an association.

invariant Any condition on the objects of a conceptual model, their attribute values and links that must hold for the requirements to be satisfied and the consistency of the model to be guaranteed.

link A link exists between two objects when there is a connection between the entities they represent that is significant for the system domain. A link is an instance of an association.

multiplicity The multiplicity of an association with respect to one of the classes involved defines how many objects of that class may be linked with a single object of the other class involved in the association.

noun phrase A noun phrase is a phrase (a sequence of words like 'number of copies') that functions as a noun and which can be used anywhere that a noun could be used.

object (in the conceptual model) An object models a real-world entity from the system domain.

organisational unit Any part of an organisational structure to which people or things in the system domain belong.

parent class A class that is a generalisation of one or more other classes known as its child classes; a superclass.

role The particular responsibility or job that a person (thing) may perform in the system domain, e.g. employee, student, lecturer, driver.

specialisation Saying that a class B is a specialisation of class A means that class A is a generalisation of class B.

stereotype A label providing additional information about some component of a UML model. Stereotypes are enclosed in double chevrons, e.g. <<abstract>>.

structural model A representation of the system in terms of classes and the relationships between them.

subclass See child class.

substitutable, substitutability Objects of a class B are substitutable for objects of a class A if anywhere that an instance of class A is required, an instance of class B can be used instead. For class A to be a generalisation of class B, instances of B must be substitutable for instances of A.

superclass See parent class.

tangible entity A physical 'thing' in the system domain such as an aeroplane, vehicle, reactor or person, i.e. something that can be seen or touched.

value (of an attribute) Specific information that an object holds in respect of an attribute.

References

Coad, P. and Yourdon, E. (1991) *Object-Oriented Analysis*, Prentice Hall PTR.

Schlaer, S. and Mellor, S. (1988) *Object-Oriented Systems Analysis: Modeling the World in Data*, Yourdon Press.

Index

A

abstract

class 25

entity 18

association 10, 12, 30

attribute 18

attributes 10

B

business area 8

C

candidate class 14

child class 23

class 10

kinds of 17

abstract 25

concrete 26

description 22

diagram 10

notation for 17

conceptual model 5, 64

concrete class 26

constraint (on the system domain) 7

core system 7, 9

D

derived

association 60

attribute 57

E

event 17

G

guidelines for rejection 15

H

Hospital System, conceptual model 64

I

inheritance 24

initial structural model 8, 64

instance 12

invariant 47

L

link 10, 30

N

notation for classes 17

O

object 10

object diagram 10–11, 30

organisational unit 18

P

parent class 23

R

real-world entity 10

rejection (guidelines for) 15

role 17

S

specialisation 23

stereotype 25

structural model 65

subclass 23

substitutability 24

superclass 23

system domain 5, 9

T

tangible entity 17

textual analysis 12

U

user interface 7

V

value (of an attribute) 10